BCPD Python Developer

All questions and correct answers

323 questions – verified bank 190, practice-exam set 133

Correct options are marked in **green**.

Part I is the verified bank – every question containing code was executed and its answer checked against CPython 3.12 before it shipped. *Part II* is the practice-exam set; four of its keys disagreed with the interpreter and are corrected here, each saying what changed. Part II contains no GUI Programming questions.

Part I — Verified bank (190)

Domain 1 — Introduction and Setup (30)

Q1 *docs*

Your cousin says "Python is an interpreted language, so it is never compiled." Which statement most accurately describes what CPython actually does with a `.py` file?

A. It translates the source directly to machine code before running anything
B. It compiles the source to bytecode, then a virtual machine executes that bytecode
C. It executes the source text one character at a time with no intermediate form
D. It requires a separate compiler to be installed before any script will run

Answer: B

CPython compiles source to bytecode and a virtual machine executes it. The bytecode step is real and observable — it is what lands in `__pycache__`. A is wrong because CPython emits no machine code. C describes no real interpreter. D is wrong because the compiler is inside the interpreter.

Q2 *docs*

After you import your own module for the first time, a new folder appears next to it. What is that folder, and what is in it?

A. `__pycache__`, holding compiled bytecode so later imports are faster
B. `__init__`, holding the package initialisation code
C. `__modules__`, holding a copy of the module source
D. `__temp__`, holding files the interpreter deletes on exit

Answer: A

`__pycache__` caches the compiled bytecode as `.pyc` so a later import skips recompiling. It appears for imported modules, not for the script you run directly. `__init__.py` is a real thing but it is a file that marks a package, not this folder.

Q3 *authored*

What is the output of this snippet, and what does it demonstrate?

```
x = 10
print(type(x).__name__)
x = "ten"
print(type(x).__name__)
```

A. `int` then `str` — Python is dynamically typed, so a name can be rebound to any type
B. `int` then `int` — once a name holds an integer it is locked to that type
C. `int` then `TypeError` — reassigning to another type raises
D. `str` then `str` — Python infers one type for the whole program

Answer: A

Python is dynamically typed: the type belongs to the object, not the name, so rebinding `x` to a string is legal. This is also why type errors surface at runtime rather than before the program starts.

Q4 *docs*

What is PEP 8?

- **A.** The official style guide for Python code — naming, indentation, line length, spacing
- B. A module in the standard library that reformats source files
- C. The specification of the Python bytecode format
- D. The proposal that introduced the `print` function

Answer: A

PEP 8 is the style guide for Python code — 4-space indents, `snake_case` for functions and variables, `CapWords` for classes. It is a convention document, not code. Tools like `flake8` check against it but are not PEP 8 itself.

Q5 *authored*

Which two statements about comments in Python are true?

- **A.** # starts a comment that runs to the end of the line
- B. Python has a dedicated block-comment syntax written `/* ... */`
- **C.** A triple-quoted string on its own line is a string expression, not a comment
- D. Comments are stored in the compiled bytecode and readable at runtime

Answer: A, C

 # runs to end of line and is the only comment syntax — Python has no `/* */`. A triple-quoted string on its own line is an expression that is evaluated and discarded, which is why it works as a docstring in the right position but is not a comment. Comments are discarded at compile time.

Q6 *authored*

Which of these is a valid variable name in Python?

- A. `2nd_score`
- **B.** `_total`
- C. `class`
- **D.** `total_2`

Answer: B, D

A name may contain letters, digits and underscores but may not start with a digit, so `2nd_score` is a syntax error. `class` is a reserved keyword. Leading underscores are legal and conventionally mean "internal".

Q7 *docs*

You want to confirm which Python version is installed before starting a project. Which command does that from a terminal?

- **A.** `python -version`
- B. `python -check`
- C. `pip version`
- D. `python -v`

Answer: A

`python -version` (or `-V`) prints the version. Lowercase `-v` is verbose mode and floods the terminal with import tracing — a genuinely confusing thing to run by accident.

Q8 *docs*

What problem does a virtual environment (`python -m venv .venv`) solve?

- **A.** It gives the project its own isolated set of installed packages, so two projects can need different versions of the same library without conflict
- B. It compiles the project to a single executable file
- C. It runs the project in a sandbox with no access to the filesystem
- D. It automatically installs every package the code imports

Answer: A

A virtual environment gives the project its own `site-packages`, so project A can use one version of a library and project B another. It does not sandbox the filesystem and it does not install anything on its own.

Q9 *docs*

Which command installs the `requests` package from PyPI?

A. `pip install requests`
B. `python install requests`
C. `import requests -install`
D. `pip get requests`

Answer: A

`pip install <name>` fetches from PyPI. Inside a project prefer `python -m pip install <name>`, which guarantees the pip belonging to the Python you are actually running.

Q10 *authored*

What is the difference between the interactive interpreter (the REPL) and running a script?

A. In the REPL an expression's value is echoed automatically; in a script you must `print()` it
B. The REPL cannot define functions
C. A script cannot import modules
D. There is no difference — the REPL runs the file line by line

Answer: A

The REPL echoes the value of any expression you type; a script does not, so a script that computes without printing appears to do nothing. That surprise is the single most common first-week confusion.

Q11 *docs*

A file `hello.py` sits in the current directory. Which command runs it?

A. `python hello.py`
B. `run hello.py`
C. `python -c hello.py`
D. `execute hello.py`

Answer: A

`python hello.py` runs the file. If `python` is not found, the launcher `py hello.py` on Windows or `python3 hello.py` on macOS and Linux usually is.

Q12 *mined*

A beginner writes this and is surprised when it fails. What is the value and type of `age` after the first line, and what happens next?

```
age = input("Age? ")
result = age + 1
```

A. `age` is a `str`, so `age + 1` raises `TypeError` — `input()` always returns a string
B. `age` is an `int`, so the code works and `result` is the age plus one
C. `age` is a `float`, so `result` is a float
D. `age` is a `str`, but Python converts it automatically, so `result` works

Answer: A

`input()` always returns a string, even when the user types digits, so `"25" + 1` raises `TypeError: can only concatenate str`. Wrap it: `age = int(input("Age? "))`. Python never converts types silently for `+` between `str` and `int`.

Q13 *authored*

What is the value of a and b?

```
a = -7 // 2
b = -7 % 2
print(a, b)
```

A. `-4 1`
B. `-3 -1`
C. `-3 1`
D. `-4 -1`

Answer: A

`//` floors toward negative infinity, so `-7 // 2` is `-4`, not `-3`. Python's `%` then returns a result with the sign of the divisor, so `-7 % 2` is `1`. This differs from C and Java and is a reliable trap.

Q14 *authored*

What does this print?

```
print(2 ** 3 ** 2)
```

A. `512`
B. `64`
C. `36`
D. `128`

Answer: A

`**` is right-associative, so this is `2 ** (3 ** 2)` = `2 ** 9` = `512`. Left-associative evaluation would give `8 ** 2` = `64`, which is option B and the answer most people pick.

Q15 *authored*

What is the final value of n?

```
n = 5
n += 3
n *= 2
n -= 1
n //= 3
print(n)
```

A. `5`
B. `4`
C. `15`
D. `6`

Answer: A

Step by step: `5`, then `+= 3` gives `8`, `*= 2` gives `16`, `-= 1` gives `15`, `//= 3` gives `5`. Augmented assignment applies the operator and rebinds the name.

Q16 *authored*

What does this print?

```
x, y = 1, 2
x, y = y, x
print(x, y)
```

A. `2 1`
B. `1 2`
C. `2 2`
D. `1 1`

Answer: A

The right-hand side `y, x` is built into a tuple first, then unpacked into the left, so the swap needs no temporary variable. This is idiomatic Python and worth recognising on sight.

Q17 *authored*

You have the string `"12"` and want to add 3 to it numerically. Which expression gives 15?

A. `int("12") + 3`
B. `"12" + 3`
C. `str("12") + 3`
D. `"12" + str(3)`

Answer: A

`int("12")` converts the string to a number, then `+ 3` is arithmetic. `"12" + 3` raises `TypeError`; `"12" + str(3)` gives the string `"123"`, which is concatenation, not addition.

Q18 *docs*

Which expression produces the string `Total: 7`?

```
n = 7
```

A. `f"Total: {n}"`
B. `"Total: n"`
C. `f"Total: n"`
D. `"Total: " + n`

Answer: A

An f-string substitutes the expression inside `{}`. Without the `f` prefix the braces are literal characters, which is why option C prints `Total: n`. Option D fails because you cannot concatenate `str` and `int`.

Q19 *authored*

What does this print?

```
x = 5
print(1 < x < 10)
```

A. `True`
B. `False`
C. `1`
D. it raises a `SyntaxError`

Answer: A

Python allows chained comparison: `1 < x < 10` means `1 < x and x < 10`, evaluating x once. Most languages parse this as `(1 < x) < 10` and get a nonsense result.

Q20 *mined*

What is the value of `i` after this runs?

```
i = 0
while i != 0:
    i = i - 1
else:
    i = i + 1
```

A. `1`
B. `0`
C. `2`
D. the variable becomes unavailable outside the loop

Answer: A

`while ... else` runs the `else` block when the loop finishes without `break`. Here the condition is false immediately, so the body never runs, the loop ends normally, and `else` executes `i = i + 1`, giving 1. Practice sites key this question three different ways and one popular copy renders `!=` with a space as `! =`, which would be a `SyntaxError` and is a typesetting artifact, not the question.

Q21 *authored*

What does this print?

```
for n in range(5):
    if n == 3:
        break
else:
    print("finished")
print("done")
```

A. done
B. `finished` then done
C. done then `finished`
D. `finished` only

Answer: A

`for ... else` runs the `else` only if the loop was never broken out of. The `break` at `n == 3` fires, so `finished` never prints. Read `else` here as "no break".

Q22 *docs*

What does `list(range(2, 10, 3))` produce?

A. `[2, 5, 8]`
B. `[2, 5, 8, 11]`
C. `[3, 6, 9]`
D. `[2, 4, 6, 8]`

Answer: A

`range(start, stop, step)` excludes the stop value. From 2 in steps of 3: 2, 5, 8 — the next would be 11, which is past 10. The exclusive endpoint is the standard off-by-one source.

Q23 *authored*

What does this print?

```
total = 0
for n in range(6):
    if n % 2 == 0:
        continue
    if n > 4:
        break
    total += n
print(total)
```

A. 4
B. 9
C. 6
D. 1

Answer: A

`continue` skips even numbers, so only 1, 3, 5 reach the second test. `break` fires at 5 before it is added. Total is 1 + 3 = 4.

Q24 *docs*

What is `pass` for?

A. It is a statement that does nothing, used where the syntax requires a statement but no action is wanted
B. It skips the rest of the current loop iteration
C. It exits the enclosing loop immediately
D. It returns `None` from the enclosing function

Answer: A

`pass` is the null statement — it does nothing, and exists because Python's block syntax requires at least one statement. Use it for a stub class or an empty `except`. `continue` and `break` are the loop controls.

Q25 *authored*

What does this print?

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print(i, j)
```

A. `0 0 1 0 2 0`
B. `0 0` only
C. `0 0 0 1 0 2`
D. nothing

Answer: A

`break` exits only the innermost loop, so each outer iteration prints once at `j == 0` then breaks. To leave both loops you need a flag, a `return`, or an `else` clause on the outer loop.

Q26 *authored*

A grading script gives the wrong letter for a score of 95. What is wrong?

```
score = 95
if score >= 50:
    grade = "Pass"
elif score >= 90:
    grade = "Distinction"
else:
    grade = "Fail"
print(grade)
```

A. The first true branch wins, so `score >= 50` matches first and the Distinction branch can never be reached — order the conditions from most specific to least
B. `elif` cannot follow a branch that assigned a variable
C. `score >= 90` is false for 95
D. The `else` branch always runs last regardless

Answer: A

An `if/elif` chain stops at the first true test. `95 >= 50` is true, so `grade` becomes `Pass` and the Distinction branch is unreachable for every score. Order the tests from most specific to least, or test `score >= 90` first.

Q158 *authored*

A function is supposed to start with an empty list every call. Why does the second call print two items?

```
def collect(item, bucket=[]):
    bucket.append(item)
    return bucket

print(collect("a"))
print(collect("b"))
```

A. The default `[]` is created once when the function is defined, so every call that omits `bucket` shares the same list — use `bucket=None` and build the list inside
B. `append` returns a new list each time
C. Python caches the return value of the first call
D. The second call reuses the first call's argument by name

Answer: A

A default argument is evaluated once, when the `def` runs — not on each call. So every call that omits `bucket` appends to the same list, and it grows for the life of the program. The fix is `def collect(item, bucket=None):` then `if bucket is None: bucket = []`. This is the single most-cited Python gotcha and it is worth recognising instantly.

Q159 *authored*

Two lists hold the same values. Why does one comparison print `True` and the other `False`?

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b)
```

A. `==` compares the values, `is` compares identity — they are equal lists but two separate objects
B. `is` is a faster spelling of `==` and this output is a bug
C. a and b point at the same object, so both should be `True`
D. Lists cannot be compared with `==`

Answer: A

`==` asks "same value", `is` asks "same object in memory". Two separately built lists are equal but not identical. Use `is` only for `None`, `True` and `False`; everywhere else you almost always mean `==`.

Q160 *authored*

A price check keeps failing even though the arithmetic looks right. What is going on?

```
total = 0.1 + 0.2
print(total == 0.3, round(total, 2) == 0.3)
```

A. Floats are stored in binary and `0.1 + 0.2` is very slightly more than `0.3`, so compare with rounding or a tolerance rather than `==`
B. Python cannot add two floats
C. `0.3` is being read as a string
D. `round()` is broken for two decimal places

Answer: A

Binary floating point cannot represent 0.1 or 0.2 exactly, so the sum is 0.30000000000000004. Never compare floats with `==`; round, or test that the difference is below a small tolerance. Money is better handled with `decimal.Decimal` or whole pence as integers.

Q161 *authored*

A loop that should print three lines prints only one. What is wrong?

```
for n in [1, 2, 3]:
    print(n)
    break
```

A. `break` leaves the loop on the first pass — `continue` is what skips to the next item
B. `print` consumes the rest of the list
C. The list needs to be wrapped in `range()`
D. `break` is only valid inside `while`

Answer: A

`break` ends the whole loop on the first iteration. `continue` is the one that skips the rest of the current pass and moves to the next item. Mixing them up is a very common first-week error.

Domain 2 — Data Structures (22)

Q27 *authored*

What does this print?

```
nums = [1, 2, 3]
more = nums
more.append(4)
print(nums)
```

A. `[1, 2, 3, 4]` — both names refer to the same list object
B. `[1, 2, 3]` — `more` is a copy

C. `[4]`
D. it raises a `TypeError`

Answer: A

Assignment binds another name to the same object; it does not copy. Mutating through either name is visible through both. This aliasing is the most common beginner data-structure bug.

Q28 *authored*

You want `copy` to be an independent list so appending to it leaves `nums` unchanged. Which two do that?

A. `copy = nums[:]`
B. `copy = nums`
C. `copy = list(nums)`
D. `copy = nums.append`

Answer: A, C

Both `nums[:]` and `list(nums)` build a new list. `copy = nums` aliases. `nums.append` without parentheses binds the method object itself, which is legal but does nothing useful. Both copies here are shallow — nested objects are still shared.

Q29 *docs*

What does this print?

```
letters = ["a", "b", "c", "d", "e"]
print(letters[1:4])
```

A. `['b', 'c', 'd']`
B. `['b', 'c', 'd', 'e']`
C. `['a', 'b', 'c', 'd']`
D. `['b', 'c']`

Answer: A

Slicing includes the start index and excludes the stop, so `[1:4]` gives indices 1, 2 and 3.

Q30 *authored*

What does this print?

```
nums = [3, 1, 2]
result = nums.sort()
print(result)
```

A. None — `list.sort()` sorts in place and returns None
B. `[1, 2, 3]`
C. `[3, 1, 2]`
D. it raises an `AttributeError`

Answer: A

`list.sort()` mutates in place and returns None, so assigning its result throws away the list and keeps None. The pattern `nums = nums.sort()` silently destroys data and is worth recognising.

Q31 *docs*

You need the sorted values but must leave the original list untouched. Which do you use?

A. `sorted(nums)`
B. `nums.sort()`
C. `nums.sorted()`
D. `reverse(nums)`

Answer: A

`sorted()` is the built-in that returns a new sorted list and leaves the original alone. There is no `nums.sorted()` method, and `reverse()` is a list method that reverses in place.

Q32 *authored*

What does this print?

```
nums = [1, 2, 3, 4]
print([n * 2 for n in nums if n % 2 == 0])
```

A. `[4, 8]`
B. `[2, 4, 6, 8]`
C. `[2, 6]`
D. `[4, 8, 12, 16]`

Answer: A

The comprehension filters first with the `if`, then applies the expression, so only 2 and 4 survive and become 4 and 8.

Q33 *authored*

Which two statements about tuples are true?

A. A tuple is immutable — you cannot add, remove or replace its elements
B. A tuple can be used as a dictionary key if all its elements are hashable
C. A tuple has an `append()` method
D. Tuples cannot contain elements of different types

Answer: A, B

Tuples are immutable, which is exactly what makes them hashable and therefore usable as dictionary keys when their contents are hashable too. They have no `append()`, and they can hold mixed types freely.

Q34 *authored*

What is the type of `x`?

```
x = (5)
```

A. `int` — parentheses alone do not make a tuple; a trailing comma does
B. `tuple`
C. `list`
D. `str`

Answer: A

The comma makes a tuple, not the parentheses. `(5)` is just 5 in brackets; `(5,)` is a one-element tuple. This bites when returning a single value you meant to be a tuple.

Q35 *authored*

What does this print?

```
t = (1, 2, 3)
try:
    t[0] = 9
    print("changed")
except TypeError:
    print("immutable")
```

A. `immutable`
B. `changed`
C. nothing
D. it raises an unhandled `TypeError`

Answer: A

Item assignment on a tuple raises `TypeError: 'tuple' object does not support item assignment`, which the handler catches. Note it is `TypeError`, not `AttributeError`.

Q36 *docs*

What does this print?

```
person = {"name": "Ada", "age": 36}
print(person.get("email", "none"))
```

A. none — `get()` returns the default instead of raising when the key is missing
B. it raises a `KeyError`
C. `None`
D. `email`

Answer: A

`get(key, default)` returns the default when the key is missing instead of raising. With no default it returns `None`.

Q37 *docs*

What is the difference between `person["email"]` and `person.get("email")` when the key is absent?

A. The subscript raises `KeyError`; `get()` returns `None`
B. Both return `None`
C. Both raise `KeyError`
D. The subscript returns `None`; `get()` raises `KeyError`

Answer: A

Subscripting a missing key raises `KeyError`; `get()` returns `None`. Choose the subscript when a missing key is a bug you want to hear about, and `get()` when absence is normal.

Q38 *authored*

What does this print?

```
scores = {"a": 1, "b": 2}
scores["c"] = 3
scores["a"] = 9
print(len(scores), scores["a"])
```

A. `3 9`
B. `4 1`
C. `3 1`
D. `4 9`

Answer: A

Assigning a new key adds it; assigning an existing key replaces the value. So the dict has three keys and `"a"` is 9.

Q39 *docs*

Which loop prints each key with its value?

```
scores = {"a": 1, "b": 2}
```

A. `for k, v in scores.items():  print(k, v)`
B. `for k, v in scores:  print(k, v)`
C. `for k in scores.values():  print(k, scores[k])`
D. `for k, v in scores.keys():  print(k, v)`

Answer: A

`.items()` yields `(key, value)` pairs, which unpack into `k, v`. Iterating the dict directly yields keys only, so B fails to unpack, and `.values()` yields values, so C indexes with the wrong thing.

Q40 *authored*

What does this print?

```
nums = [1, 2, 2, 3, 3, 3]
print(len(set(nums)))
```

A. 3
B. 6
C. 1
D. 2

Answer: A

A set discards duplicates, leaving `{1, 2, 3}` — length 3. `len(set(x))` is the idiomatic way to count distinct values.

Q41 *docs*

What do these two sets produce?

```
a = {1, 2, 3}
b = {3, 4}
print(a & b, a | b)
```

A. `{3} {1, 2, 3, 4}`
B. `{1, 2} {3}`
C. `{3, 4} {1, 2}`
D. it raises a `TypeError`

Answer: A

`&` is intersection (members of both) and `|` is union (members of either). `-` gives difference and `^` symmetric difference.

Q42 *authored*

Which two statements about sets are true?

A. A set stores no duplicate values
B. A set does not support indexing such as `s[0]`
C. A set preserves the insertion order of its elements
D. A set can contain a list as an element

Answer: A, B

Sets hold no duplicates and are unordered, so there is no indexing and no reliable insertion order. Elements must be hashable, which is why a list cannot go in a set but a tuple can.

Q43 *authored*

What is the type of `x`, and why does it catch people out?

```
x = {}
```

A. `dict` — `{}` is an empty dictionary; an empty set must be written `set()`
B. `set`
C. `list`
D. `tuple`

Answer: A

`{}` is an empty dictionary — dictionaries got the braces first. An empty set has to be `set()`. Non-empty `{1, 2}` is a set, which makes the empty case inconsistent-looking and easy to get wrong.

Q44 *authored*

A stock system must hold the ordered sequence of every scan, allow duplicates, and be edited in place. Which structure fits?

A. A list
B. A set

C. A tuple
D. A frozenset

Answer: A

Ordered, duplicates allowed and mutable is the definition of a list. A set loses order and duplicates, a tuple cannot be edited, and a frozenset is an immutable set.

Q162 *authored*

A script removes every 2 from a list, but one survives. Why?

```
nums = [1, 2, 2, 3]
for n in nums:
    if n == 2:
        nums.remove(n)
print(nums)
```

A. Removing from a list while iterating it shifts the remaining items back, so the loop skips one — build a new list instead
B. `remove()` deletes only the last match
C. The loop stops as soon as it removes anything
D. `remove()` needs an index, not a value

Answer: A

The loop walks by position while `remove()` shifts everything left, so after removing index 1 the loop moves to index 2 and steps straight over the second 2. Iterate over a copy (`for n in nums[:]`) or build a new list with a comprehension.

Q163 *authored*

What happens here, and what is the fix?

```
scores = {"a": 1, "b": 2}
try:
    for k in scores:
        if k == "a":
            del scores[k]
    print(scores)
except RuntimeError as e:
    print("RuntimeError")
```

A. `RuntimeError` — a dictionary cannot change size while it is being iterated; iterate over `list(scores)` instead
B. `{'b': 2}` — deleting during iteration is fine
C. `KeyError`
D. it loops forever

Answer: A

Adding or deleting keys during iteration raises `RuntimeError: dictionary changed size during iteration`. Iterate over a snapshot — `for k in list(scores)` — or collect the keys to delete and remove them afterwards.

Q164 *authored*

A tuple is meant to be unchangeable, so why does this work — and why does the second line still fail?

```
t = (1, [2, 3])
t[1].append(4)
print(t)
try:
    {t}
except TypeError:
    print("unhashable")
```

A. The tuple's own contents are fixed, but a list stored inside it can still be changed — and that mutability is why the tuple cannot be hashed
B. Tuples are fully mutable
C. `append` on a tuple element raises `TypeError`
D. The tuple becomes a list after the append

Answer: A

*A tuple fixes *which objects* it holds, not whether those objects can change. The inner list is still mutable. And because that list is unhashable, the whole tuple becomes unhashable too, so it cannot go in a set or be a dictionary key.*

Q165 *authored*

Why does building this set fail, and what fixes it?

```
try:
    tags = {["a", "b"], ["c"]}
except TypeError:
    tags = {("a", "b"), ("c",)}
print(len(tags))
```

A. Set members must be hashable and a list is not — use tuples
B. A set cannot hold more than one item of the same length
C. Sets cannot hold sequences at all
D. The braces make a dictionary, so it needs keys

Answer: A

Set members must be hashable, and lists are not because they can change. Convert to tuples. The same rule governs dictionary keys — note `("c",)` needs the trailing comma to be a tuple.

Domain 3 — Object-Oriented Programming (35)

Q45 *docs*

What is the relationship between a class and an object?

A. A class is the blueprint; an object is a concrete instance built from it
B. An object is the blueprint; a class is one instance of it
C. They are two names for the same thing
D. A class can exist only inside an object

Answer: A

A class describes structure and behaviour; an object is one concrete thing built from it. `Dog` is the class, `Rex` is the object.

Q46 *authored*

Which of these is *not* one of the commonly listed pillars of object-oriented programming?

A. Compilation
B. Encapsulation
C. Inheritance
D. Polymorphism

Answer: A

Compilation is a build step, not an OOP principle. The pillars usually listed are encapsulation, inheritance, polymorphism and abstraction.

Q47 *authored*

What does this print?

```
class Dog:
    species = "canine"

    def __init__(self, name):
        self.name = name

a = Dog("Rex")
b = Dog("Bo")
print(a.species, b.species, a.name, b.name)
```

A. `canine canine Rex Bo`
B. `canine canine Bo Bo`

C. `Rex Bo Rex Bo`
D. it raises an `AttributeError`

Answer: A

`species` is defined in the class body, so it is a class attribute shared by every instance. `name` is assigned on `self` in `__init__`, so each instance has its own.

Q48 *authored*

What does this print, and why is it a classic trap?

```
class Counter:
    total = 0

    def bump(self):
        Counter.total += 1

x, y = Counter(), Counter()
x.bump()
y.bump()
print(x.total, y.total)
```

A. `2 2` - `total` is a class variable shared by every instance
B. `1 1` - each instance gets its own copy
C. `1 2`
D. `0 0`

Answer: A

`Counter.total` names the class attribute, so both calls increment the same counter and both instances read the same value. Had `bump` used `self.total += 1` it would have created a separate instance attribute on first assignment and printed `1 1`.

Q49 *authored*

What does this print?

```
class Box:
    def __init__(self):
        self.items = []

p = Box()
q = Box()
p.items.append("nail")
print(len(p.items), len(q.items))
```

A. `1 0` - each instance gets its own `items` list because it is created in `__init__`
B. `1 1`
C. `0 0`
D. `2 0`

Answer: A

The list is created fresh inside `__init__` on each instantiation, so it belongs to the instance. Had `items = []` been written in the class body instead, every instance would share one list - the classic mutable-class-attribute bug.

Q50 *authored*

What does this print?

```
class Point:
    pass

a = Point()
b = Point()
c = a
print(a is b, a is c)
```

A. `False True`

B. `True True`
C. `False False`
D. `True False`

Answer: A

Each call to `Point()` builds a distinct object, so `a is b` is false. `c = a` binds another name to the same object, so `a is c` is true. `is` compares identity, == compares value.

Q51 *docs*

Which two statements about `__init__` are true?

A. It is called automatically straight after the new instance is created
B. It is the method that allocates and returns the new object
C. Its first parameter is the instance, conventionally named `self`
D. A class is invalid without an explicit `__init__`

Answer: A, C

`__init__` initialises an already-created instance and receives it as `self`. Allocation is `__new__`'s job. A class with no `__init__` inherits `object.__init__` and works fine.

Q52 *authored*

What happens here?

```
class Item:
    def __init__(self):
        return 42

try:
    Item()
    print("built")
except TypeError:
    print("TypeError")
```

A. `TypeError` - `__init__` must return None
B. `built`
C. `42`
D. nothing is printed

Answer: A

`__init__` must return None; returning anything else raises `TypeError: __init__() should return None`. Set attributes on `self` instead of returning a value.

Q53 *authored*

What does this print?

```
class User:
    def __init__(self, name, role="member"):
        self.name = name
        self.role = role

a = User("Ada")
b = User("Bo", "admin")
print(a.role, b.role)
```

A. `member admin`
B. `admin admin`
C. `member member`
D. it raises a `TypeError` because `User("Ada")` is missing an argument

Answer: A

A default value makes the parameter optional, so `User("Ada")` uses `member`. This is the Python answer to "constructors with different signatures" - one constructor with defaults, not several.

Q54 *docs*

What does this print?

```
class Car:
    def __init__(self):
        self.wheels = 4

c = Car()
print(hasattr(c, "wheels"), hasattr(c, "wings"), getattr(c, "wings", 0))
```

A. `True False 0`
B. `True True 0`
C. `True False None`
D. it raises an `AttributeError`

Answer: A

`hasattr` reports whether the attribute exists. `getattr(obj, name, default)` returns the default rather than raising when it does not. Without the default it would raise `AttributeError`.

Q55 *authored*

What does this print?

```
class Base:
    tag = "class"

b = Base()
print(b.tag)
b.tag = "instance"
print(b.tag, Base.tag)
```

A. `class` then `instance class`
B. `class` then `instance instance`
C. `class` then `class class`
D. it raises an `AttributeError`

Answer: A

Attribute lookup checks the instance first, then the class. Assigning `b.tag` creates an instance attribute that shadows the class attribute for that object only - `Base.tag` is untouched, and other instances still see `class`.

Q56 *authored*

What does an instance's `__dict__` hold?

```
class P:
    kind = "shape"
    def __init__(self):
        self.x = 1

print(sorted(P().__dict__))
```

A. `['x']` - only the instance attributes, not the class attributes
B. `['kind', 'x']`
C. `['kind']`
D. `[]`

Answer: A

An instance's `__dict__` holds only what was set on the instance. Class attributes live in `P.__dict__`, which is why lookup has to check both.

Q57 *docs*

In `car = Car("blue")`, what is `Car("blue")` called, and what is `car`?

A. Instantiation, and `car` is an instance of `Car`
B. Inheritance, and `car` is a subclass of `Car`
C. Overriding, and `car` is a method
D. Encapsulation, and `car` is an attribute

Answer: A

Calling a class instantiates it and returns an instance. The vocabulary matters on this exam: class, instance, attribute, method, instantiation.

Q58 *docs*

Why does every instance method take `self` as its first parameter?

A. Python passes the instance explicitly as the first argument; `self` is the name that receives it
B. `self` is a reserved keyword that Python fills in automatically
C. It is optional and can be left out
D. It refers to the class, not the instance

Answer: A

`obj.method()` is shorthand for `Class.method(obj)`, so the instance arrives as the first positional argument. `self` is a naming convention, not a keyword - but breaking it will confuse every reader.

Q59 *docs*

What does encapsulation mean in practice?

A. Bundling data and the methods that operate on it into one unit, and controlling access to the internals
B. Allowing a class to inherit from more than one parent
C. Letting different types respond to the same method call
D. Preventing a class from ever being subclassed

Answer: A

Encapsulation bundles state with the behaviour that acts on it and keeps the internals out of the public surface. Option B describes multiple inheritance and C describes polymorphism.

Q60 *docs*

A colleague names an attribute `self._cache`. What does the single leading underscore mean?

A. It is a convention meaning "internal, do not rely on this" - Python does not enforce it
B. Python makes the attribute unreadable from outside the class
C. It makes the attribute read-only
D. It renames the attribute at runtime

Answer: A

One underscore is a documented convention meaning "not part of the public API". Nothing enforces it - the attribute is fully readable. PEP 8 describes it as a weak internal-use indicator.

Q61 *authored*

What does this print?

```
class Animal:
    def speak(self):
        return "..."

class Dog(Animal):
    pass

print(Dog().speak())
```

A. `...` - Dog inherits `speak` from `Animal`

B. it raises an `AttributeError`
C. `None`
D. `speak`

Answer: A

`Dog` defines nothing, so lookup walks up to `Animal` and finds `speak`. Inheritance means the subclass gets the parent's methods without repeating them.

Q62 *docs*

What does this print?

```
class Vehicle:
    def __init__(self, wheels):
        self.wheels = wheels

class Bike(Vehicle):
    def __init__(self):
        super().__init__(2)
        self.pedals = True

b = Bike()
print(b.wheels, b.pedals)
```

A. `2 True`
B. `0 True`
C. it raises a `TypeError` because `Bike()` takes no arguments
D. it raises an `AttributeError` because `wheels` is never set

Answer: A

The subclass's `__init__` overrides the parent's, so the parent's must be called explicitly. `super().__init__(2)` does that, setting `wheels` before the subclass adds `pedals`. Forgetting the `super()` call is how half-initialised objects happen.

Q63 *docs*

What does this print?

```
class A: pass
class B(A): pass

b = B()
print(isinstance(b, A), issubclass(B, A), isinstance(A(), B))
```

A. `True True False`
B. `False True False`
C. `True True True`
D. `True False False`

Answer: A

`isinstance(b, A)` is true because B derives from A, and `issubclass(B, A)` is true for the same reason. An A is not a B, so the third is false - inheritance runs one way only.

Q64 *authored*

What does this print?

```
class A:
    def who(self):
        return "A"

class B(A):
    def who(self):
        return "B"

class C(A):
    def who(self):
```

```
        return "C"

class D(B, C):
    pass

print(D().who())
```

A. B
B. C
C. A
D. it raises a `TypeError` because of the ambiguous inheritance

Answer: A

Python resolves multiple inheritance by the method resolution order, which for `D(B, C)` is D, B, C, A. `B.who` is found first. Python does not raise on the diamond; it linearises it.

Q65 *authored*

What does this print?

```
class Shape:
    def area(self):
        return 0

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

print(Square(3).area())
```

A. 9 - the subclass method overrides the parent's
B. `0`
C. `3`
D. it raises a `TypeError`

Answer: A

`Square.area` overrides `Shape.area` because the subclass is searched first. This is the mechanism behind polymorphism: same call, different behaviour by type.

Q66 *docs*

What does this print?

```
class Coin:
    def __init__(self, value):
        self.value = value
    def __str__(self):
        return f"{self.value}p"

print(Coin(50))
```

A. `50p` - `print()` converts its argument with `str()`, which calls `__str__`
B. the default `<__main__.Coin object at 0x...>` representation
C. `50`
D. it raises a `TypeError`

Answer: A

`print()` converts its argument with `str()`, which calls `__str__`. Without it you get the default `<Coin object at 0x...>`. `__repr__` is the developer-facing sibling and is what the REPL shows.

Q67 *authored*

What does this print?

```
class Vault:
    def __init__(self):
        self.__code = 1234

v = Vault()
print(hasattr(v, "__code"), v._Vault__code)
```

A. `False 1234` - the double underscore triggers name mangling to `_Vault__code`
B. `True 1234`
C. it raises an `AttributeError` on the second expression
D. `False 0`

Answer: A

An attribute starting with two underscores is name-mangled to `_ClassName__attr`, so `v.__code` fails from outside while `v._Vault__code` works. The purpose is avoiding accidental clashes in subclasses, not security.

Q68 *docs*

Which statement about "private" attributes in Python is true?

A. There is no enforced privacy - a double underscore mangles the name, which discourages access but does not prevent it
B. A double-underscore attribute is encrypted in memory
C. `private` is a keyword that blocks external access
D. Single-underscore attributes raise `AttributeError` when accessed from outside

Answer: A

Python has no access modifiers. Double underscore mangles the name, single underscore signals intent, and neither blocks access. The documented model is that everything is reachable and conventions carry the meaning.

Q69 *authored*

What does this print, and what does it say about function overloading in Python?

```
class Calc:
    def add(self, a, b):
        return a + b
    def add(self, a, b, c):
        return a + b + c

print(Calc().add(1, 2, 3))
```

A. 6 - the second `def` replaces the first, so Python has no function overloading
B. 3 - Python picks the two-argument version when given two arguments
C. it raises a `TypeError` for a duplicate method name
D. `1`

Answer: A

Defining a method twice simply rebinds the name - the second definition wins and the first is gone, so a two-argument call would now fail. This is why the blueprint's phrase "function overloading" does not describe real Python behaviour. See the objectives note on 3.11.

Q70 *authored*

What does this print, and which OOP idea does it show?

```
class Cat:
    def speak(self):
        return "meow"

class Duck:
    def speak(self):
        return "quack"
```

```
for animal in (Cat(), Duck()):
    print(animal.speak())
```

A. meow then quack - polymorphism through duck typing; no shared base class is needed
B. it raises a TypeError because the classes are unrelated
C. meow twice
D. quack twice

Answer: A

Both objects answer speak(), so the loop works without a common base class. Python cares that the method exists, not what the type is - duck typing, and the usual meaning of polymorphism here.

Q71 *docs*

Python has no function overloading. Which two are the idiomatic ways to let one method accept a varying number of arguments?

A. Give parameters default values
B. Collect extras with *args
C. Define the method twice with different parameter lists
D. Annotate the method with @overload so Python dispatches at runtime

Answer: A, B

*Defaults and *args are how one Python function accepts varying calls. Defining it twice replaces the first definition. typing.overload exists but is a type-checker hint with no runtime effect.*

Q166 *authored*

Two carts are created and one item is added, but both carts show it. What is wrong?

```
class Cart:
    items = []
    def add(self, thing):
        self.items.append(thing)

a, b = Cart(), Cart()
a.add("apple")
print(b.items)
```

A. items is a class attribute shared by every instance — create it per instance with self.items = [] inside __init__
B. a and b are the same object
C. append writes to every list in the program
D. self.items is a typo for Cart.items

Answer: A

items = [] in the class body creates one list shared by every instance, so both carts see the apple. Assign it in __init__ with self.items = [] and each instance gets its own. A mutable class attribute is the OOP twin of the mutable default argument in Q158.

Q167 *authored*

A record has data but no behaviour, and is only ever read. What is the reasonable design call?

A. A dictionary or a small data class is enough — a class earns its place when data and behaviour belong together, not for grouping values alone
B. Always use a class; dictionaries are never appropriate
C. Always use a dictionary; classes are only for inheritance
D. Use a tuple, because classes cannot hold read-only data

Answer: A

Classes earn their place when data and the behaviour that acts on it belong together. Values with no behaviour are usually better as a dictionary or a small data class. "Use a class for everything" produces classes that are dictionaries with extra typing.

Q168 *authored*

Why does the second instance not see the change?

```
class Config:
    debug = False

a, b = Config(), Config()
a.debug = True
print(a.debug, b.debug, Config.debug)
```

A. Assigning through the instance creates a new instance attribute that shadows the class attribute for that object only
B. `Config.debug` is copied to each instance at creation
C. a and b are the same object
D. it raises an `AttributeError`

Answer: A

*Reading falls back to the class, but *writing* always creates an instance attribute. So `a.debug = True` shadows the class attribute for a alone, and b and `Config` are untouched. Set it on the class (`Config.debug = True`) if you meant it globally.*

Q169 *docs*

In the line `report.export("csv")`, name the parts in order: `report`, `export`, and `"csv"`.

A. An instance, a method, and an argument
B. A class, an attribute, and a parameter
C. A module, a function, and a return value
D. An object, a constructor, and an instance

Answer: A

`report` is an instance, `export` is a method on its class, and `"csv"` is the argument passed to that method. The parameter is the name inside the method definition that receives it; the argument is the value at the call site.

Q170 *authored*

A balance must never go negative, but this lets it. What is the encapsulation fix?

```
class Account:
    def __init__(self):
        self.balance = 0

acc = Account()
acc.balance = -50
print(acc.balance)
```

A. Keep the value internal and expose a method or `property` that validates before assigning, so the rule cannot be bypassed by writing the attribute directly
B. Rename the attribute to `__balance`, which makes it impossible to change
C. Declare `balance` as `private`
D. Nothing — a negative balance is not preventable in Python

Answer: A

Encapsulation is about controlling access, not hiding names. Keep the value internal and put the rule in a `property` setter or a method, so an invalid assignment raises rather than silently succeeding. Renaming to `__balance` only mangles the name — `acc._Account__balance = -50` still works.

Q171 *authored*

The subclass loses the parent's setup. What is missing?

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
```

```
        self.breed = breed

try:
    print(Dog("Rex", "lab").name)
except AttributeError:
    print("AttributeError")
```

A. `AttributeError` — the overriding `__init__` never calls `super().__init__(name)`, so `name` is never set
B. `Rex` — the parent's `__init__` runs automatically
C. `TypeError` — too many arguments
D. `None`

Answer: A

Defining `__init__` in the subclass replaces the parent's; it is not added to it. Without `super().__init__(name)` the parent's setup never runs and `name` is never assigned, so the attribute lookup fails at use, far from the real cause.

Q172 *authored*

A subclass sets what looks like the same private attribute, but the parent's method still returns the old value. Why?

```
class Base:
    def __init__(self):
        self.__value = 1
    def get(self):
        return self.__value

class Child(Base):
    def __init__(self):
        super().__init__()
        self.__value = 2

c = Child()
print(c.get(), c._Base__value, c._Child__value)
```

A. `1 1 2` — the double underscore is mangled per class, so `Base` and `Child` end up with two separate attributes
B. `2 2 2` — they are the same attribute
C. `1 1 1`
D. it raises an `AttributeError`

Answer: A

Name mangling is per class: `Base.__value` becomes `_Base__value` and `Child.__value` becomes `_Child__value`. They are two different attributes, so `Base.get()` still reads its own. That isolation is exactly what mangling is for — it prevents a subclass clobbering a parent's internals by accident.

Q173 *authored*

A subclass replaces a method but should still run the parent's version first. What does that?

```
class Logger:
    def write(self, msg):
        return "[log] " + msg

class Timestamped(Logger):
    def write(self, msg):
        return super().write("09:00 " + msg)

print(Timestamped().write("started"))
```

A. `[log] 09:00 started` — `super().write()` calls the parent's version from inside the override
B. `09:00 started` — the parent's version is discarded
C. it raises a `RecursionError`
D. `[log] started`

Answer: A

`super().write(...)` calls the parent implementation from inside the override, so the subclass can extend rather than replace. Calling `self.write(...)` instead would recurse forever.

Domain 4 — Modules and Libraries (36)

Q72 *docs*

After `import math`, which expression calls the square-root function?

A. `math.sqrt(9)`
B. `sqrt(9)`
C. `math->sqrt(9)`
D. `from math.sqrt(9)`

Answer: A

`import math` binds the module object to the name `math`; its contents are reached through the dot. A bare `sqrt(9)` fails with `NameError` unless it was imported directly.

Q73 *docs*

What is the difference between `import math` and `from math import sqrt`?

A. The first binds the name `math` and you call `math.sqrt()`; the second binds `sqrt` directly into your namespace
B. The first is faster because it loads less of the module
C. The second imports the whole module twice
D. There is no difference

Answer: A

Both load the module. The difference is what lands in your namespace: the module object, or the individual name. `from ... import` is not faster - the whole module still executes; only the binding differs.

Q74 *docs*

What does `import numpy as np` do?

A. It imports the module and binds it to the shorter local name np
B. It imports only the part of NumPy called np
C. It renames the installed package on disk
D. It creates a copy of the module

Answer: A

`as` renames the binding in your file only. Nothing on disk changes. `np` is a universal convention for NumPy and worth using.

Q75 *docs*

Why is `from module import *` discouraged?

A. It pulls every public name into the current namespace, where they can silently overwrite your own names
B. It is a syntax error outside a function
C. It imports the module twice
D. It prevents the module from being imported again later

Answer: A

A star import binds every public name at once, so an unnoticed collision can silently replace one of your own functions - and a reader cannot tell where a name came from. Explicit imports are the documented preference.

Q76 *authored*

What do these print?

```
import math
print(math.ceil(-2.5), math.floor(-2.5))
```

A. `-2 -3`
B. `-3 -2`
C. `-2 -2`
D. `-3 -3`

Answer: A

`ceil` rounds up toward positive infinity, so `−2.5` becomes `−2`. `floor` rounds down toward negative infinity, giving `−3`. For negative numbers "up" and "down" are the opposite of "bigger" and "smaller" in magnitude, which is the trap.

Q77 *docs*

What does this print?

```
import re
print(re.findall(r"\d+", "a1b22c333"))
```

A. `['1', '22', '333']`
B. `['1', '2', '2', '3', '3', '3']`
C. `['a', 'b', 'c']`
D. `[]`

Answer: A

`\d+` matches one or more consecutive digits, and `findall` returns every non-overlapping match as a list of strings - note strings, not integers. Without the + it would match single digits.

Q78 *docs*

What does this print?

```
from datetime import date
d = date(2026, 8, 25)
print(d.strftime("%Y-%m-%d"), d.year)
```

A. `2026-08-25 2026`
B. `25-08-2026 2026`
C. `2026-8-25 2026`
D. it raises a `ValueError`

Answer: A

`%Y` is the four-digit year, `%m` the zero-padded month and `%d` the zero-padded day. The `date` object also exposes `.year`, `.month` and `.day` directly.

Q79 *docs*

Which two statements about the `random` module are true?

A. `random.seed(42)` makes the sequence of following random numbers repeatable
B. `random.choice(seq)` returns one element drawn from `seq`
C. `random.randint(1, 6)` can never return `6`
D. `random.random()` returns an integer

Answer: A, B

*Seeding fixes the generator's starting state, so the same sequence follows - essential for reproducible tests. `randint(a, b)` is inclusive at *both* ends, so 6 is possible, and `random()` returns a float in [0.0, 1.0).*

Q80 *docs*

What is `sys.path`?

A. The list of directories the interpreter searches when importing a module
B. The path to the currently running script
C. The system PATH environment variable
D. The folder where `pip` installs packages

Answer: A

`sys.path` is the list of directories searched on import, starting with the script's own directory. Appending to it is how a script reaches modules outside its folder.

Q81 *docs*

You save some functions in `helpers.py` and want to use them from `main.py` in the same folder. What do you write in `main.py`?

A. `import helpers`
B. `include helpers.py`
C. `import helpers.py`
D. `from helpers`

Answer: A

The module name is the filename without `.py`, so `import helpers`. Writing `import helpers.py` makes Python look for a module `py` inside a package `helpers` and fails.

Q82 *docs*

What is a module, in Python's own terms?

A. A file containing Python definitions and statements, whose filename is the module name plus `.py`
B. A compiled binary that Python loads at startup
C. Any folder inside the project
D. A class with only static methods

Answer: A

The documentation defines a module as a file containing Python definitions and statements, with the file name being the module name plus the `.py` suffix.

Q83 *docs*

What does this idiom do, and why is it used?

```
def main():
    print("running")

if __name__ == "__main__":
    main()
```

A. `__name__` is `"__main__"` only when the file is run directly, so `main()` runs on execution but not on import
B. It stops the file from ever being imported
C. It renames the module to `__main__`
D. It is required in every Python file

Answer: A

Python sets `__name__` to `"__main__"` in the file being run directly, and to the module's own name when it is imported. The guard therefore separates "run me" behaviour from "import me" behaviour, so importing the file does not execute the script.

Q84 *docs*

What is the value of `__name__` inside `helpers.py` when `main.py` does `import helpers`?

A. `"helpers"`
B. `"__main__"`
C. `"main"`
D. `None`

Answer: A

On import, `__name__` is the module's name - here `"helpers"`. Only the file the interpreter was pointed at gets `"__main__"`.

Q85 *docs*

What makes a folder a Python package in the traditional layout?

A. It contains an `__init__.py` file
B. It contains a `package.json` file
C. Its name ends in `.pkg`
D. It is listed in `sys.modules`

Answer: A

`__init__.py` marks the directory as a package and runs on import. It may be empty. Namespace packages can omit it, but the exam-level answer is the `__init__.py` layout.

Q86 *docs*

Given the package layout below, which import reaches `clean()`?

```
# tools/
#     __init__.py
#     text/
#         __init__.py
#         strip.py     <- defines clean()
```

A. `from tools.text.strip import clean`
B. `import tools.text.strip` then call `tools.text.strip.clean()`
C. `import clean from tools.text.strip`
D. `from tools import clean`

Answer: A, B

Both reach the function: `from tools.text.strip import clean` binds it directly, and importing the full dotted path lets you call it through the chain. Option C reverses the syntax, and `from tools import clean` fails because `clean` is not defined in the top package.

Q87 *authored*

What does this print?

```
import numpy as np
a = np.array([1, 2, 3])
print(a)
```

A. `[1 2 3]` - NumPy prints arrays without commas
B. `[1, 2, 3]`
C. `array([1, 2, 3])`
D. `(1, 2, 3)`

Answer: A

NumPy's array display separates elements with spaces, not commas - so `[1 2 3]`. Seeing commas means it is a list. The `array([1, 2, 3])` form is the `repr`, which is what the REPL echoes.

Q88 *docs*

What does this print?

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape, a.ndim, a.size)
```

A. `(2, 3) 2 6`
B. `(3, 2) 2 6`
C. `(2, 3) 6 2`
D. `(6,) 1 6`

Answer: A

`shape` is (rows, columns), `ndim` is the number of dimensions and `size` is the total element count. Two rows of three gives `(2, 3)`, 2 and 6.

Q89 *docs*

What does this print?

```
import numpy as np
print(np.arange(0, 10, 3))
```

A. `[0 3 6 9]`

B. `[0 3 6 9 12]`
C. `[3 6 9]`
D. `[0 1 2 3 4 5 6 7 8 9]`

Answer: A

`arange(start, stop, step)` excludes the stop value, exactly like `range` - 0, 3, 6, 9, and 12 would be past 10.

Q90 *docs*

What does this print?

```
import numpy as np
print(np.zeros(3), np.ones(2, dtype=int))
```

A. `[0.  0.  0.]  [1 1]`
B. `[0 0 0] [1 1]`
C. `[0.  0.  0.]  [1.  1.]`
D. it raises a `TypeError`

Answer: A

`zeros` defaults to float, which is why they print as `0.` with a trailing dot. `dtype=int` forces the integer type, so the ones print without dots.

Q91 *authored*

What happens here, and what does it show about a NumPy array's dtype?

```
import numpy as np
a = np.array([1, 2, 3])
a[0] = 9.7
print(a)
```

A. `[9 2 3]` - the array holds one fixed type, so the float is truncated to fit the integer dtype
B. `[9.7 2.  3.  ]`
C. it raises a `TypeError`
D. `[10 2 3]`

Answer: A

An array has one dtype for every element. This array is integer, so assigning 9.7 truncates toward zero to 9 - no error, no warning, silent data loss. It is the sharpest practical difference from a list.

Q92 *docs*

Which two statements describe how a NumPy array differs from a Python list?

A. Every element shares one fixed data type
B. Arithmetic applies element by element across the whole array
C. It can hold values of mixed types as freely as a list
D. It grows with `append()` in place, exactly like a list

Answer: A, B

An array is homogeneous and typed, and arithmetic is element-wise (vectorised) rather than list concatenation. It has no `append` method - `np.append` exists but returns a new array.

Q93 *authored*

What does this print?

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b, a * 2)
```

A. `[11 22 33] [2 4 6]`
B. `[1 2 3 10 20 30] [2 4 6]`

C. `[11 22 33] [1 2 3 1 2 3]`
D. it raises a `ValueError`

Answer: A

*Both + and * work element by element on arrays. On lists + concatenates and * repeats, which is the exact confusion the next question tests.*

Q94 *authored*

A list and an array are given the same treatment. What does this print, and why do they differ?

```
nums = [1, 2, 3]
import numpy as np
arr = np.array([1, 2, 3])
print(nums * 2)
print(arr * 2)
```

A. `[1, 2, 3, 1, 2, 3]` then `[2 4 6]` - * repeats a list but multiplies an array element-wise
B. `[2, 4, 6]` then `[2 4 6]`
C. `[1, 2, 3, 1, 2, 3]` then `[1 2 3 1 2 3]`
D. it raises a `TypeError` on the second line

Answer: A

Same operator, two meanings: `list * 2` repeats the sequence, `array * 2` multiplies every element. Anyone moving from lists to NumPy hits this in the first hour.

Q95 *authored*

What does this print, and how does it differ from slicing a list?

```
import numpy as np
a = np.array([1, 2, 3, 4])
part = a[1:3]
part[0] = 99
print(a)
```

A. `[ 1 99 3 4]` - a NumPy slice is a view onto the original array, not a copy
B. `[1 2 3 4]`
C. `[99 2 3 4]`
D. it raises a `ValueError`

Answer: A

*Basic slicing of a NumPy array returns a *view* that shares memory with the original, so writing through the slice changes the parent. Slicing a list returns a copy. Use `a[1:3].copy()` when you want independence.*

Q96 *docs*

What does this print?

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a[a > 2])
```

A. `[3 4]`
B. `[False False True True]`
C. `[1 2]`
D. it raises an `IndexError`

Answer: A

`a > 2` produces a boolean array, and indexing with it selects the elements where it is True. This is boolean masking, the standard NumPy way to filter.

Q97 *authored*

What does this print?

```
import numpy as np
a = np.array([2, 4, 6, 8])
print(a.mean(), a.sum(), a.max())
```

A. `5.0 20 8`
B. `5 20 8`
C. `20 5.0 8`
D. `4.0 20 8`

Answer: A

`mean()` returns a float even for integer input, so `5.0`. `sum()` and `max()` keep the integer type.

Q98 *docs*

What does this print?

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print(a.sum(axis=0), a.sum(axis=1))
```

A. `[4 6] [3 7]`
B. `[3 7] [4 6]`
C. `[10] [10]`
D. `[1 2] [3 4]`

Answer: A

`axis=0` collapses down the rows, giving one total per column: 1+3 and 2+4. `axis=1` collapses across the columns, giving one total per row: 1+2 and 3+4. Reading the axis as "the axis that disappears" is the reliable way to remember it.

Q99 *authored*

What does this print?

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(np.median(a))
```

A. `2.5`
B. `2`
C. `3`
D. `2.0`

Answer: A

With an even number of values the median is the mean of the two middle ones, so (2+3)/2 = 2.5, and the result is a float.

Q100 *docs*

Which NumPy function returns the standard deviation of an array?

A. `np.std(a)`
B. `np.deviation(a)`
C. `np.sd(a)`
D. `np.variance(a)`

Answer: A

`np.std` is the standard deviation and `np.var` the variance. `np.deviation` and `np.sd` do not exist - plausible-looking names that were never real, which is the single most common wrong answer pattern in Python question banks.

Q174 *authored*

A student saves a practice file as `random.py` in their project folder. Their other script then fails on `random.randint(1, 6)`. Why?

A. Their own file shadows the standard library module of the same name, because the script's own directory comes first on the import path — rename the file
B. `randint` was removed from the standard library
C. Two modules cannot have the same name anywhere on a machine
D. The file must be deleted from `__pycache__` first

Answer: A

The script's own directory is first on `sys.path`, so a local `random.py` is found before the standard library's. Every `import random` in that folder then gets the wrong module. Avoid naming files after standard modules — `random.py`, `string.py`, `math.py`, `email.py` are all common casualties.

Q175 *authored*

A helper file prints a test line the moment another script imports it. What is missing?

```
# helpers.py
def add(a, b):
    return a + b

print("self test:", add(1, 2))
```

A. The test line needs an `if __name__ == "__main__":` guard so it runs only when the file is executed directly
B. `print` is not allowed in an imported module
C. The function must be defined after the print
D. The file needs to be renamed to `__main__.py`

Answer: A

Every top-level statement runs on import. The `if __name__ == "__main__":` guard confines self-tests and demo output to direct execution. Without it, importing the module for one function drags along all of its side effects.

Q176 *docs*

A folder `utils/` holds `dates.py`, but `from utils.dates import today` raises `ModuleNotFoundError`. What is the most likely cause in the traditional package layout?

A. `utils/` has no `__init__.py`, or the folder containing `utils/` is not on the import path
B. `dates.py` must be renamed `__init__.py`
C. Packages cannot contain modules
D. The import must be written `import utils/dates`

Answer: A

*In the traditional layout `utils/` needs an `__init__.py`, and the directory that *contains* `utils/` has to be on the import path — usually because it is the script's own folder. Namespace packages can go without `__init__.py`, but at this level the missing file is the likelier cause.*

Q177 *authored*

A beginner treats a NumPy array like a list and it does not work. Why does `append` fail?

```
import numpy as np
a = np.array([1, 2, 3])
b = np.append(a, 4)
print(hasattr(a, "append"), a, b)
```

A. `False [1 2 3] [1 2 3 4]` — an array has a fixed size, so there is no in-place `append`; `np.append` builds a whole new array
B. `True [1 2 3 4] [1 2 3 4]`
C. it raises an `AttributeError`
D. `False [1 2 3 4] [1 2 3 4]`

Answer: A

An array occupies a fixed block of memory with a fixed size, so there is nothing to append to in place. `np.append` allocates a new array and copies — fine occasionally, expensive in a loop. Collect into a Python list and convert once.

Q178 *authored*

Two arrays of different lengths are added and it fails. What does the error mean?

```
import numpy as np
try:
    print(np.array([1, 2, 3]) + np.array([10, 20]))
except ValueError:
    print("ValueError")
```

A. `ValueError` — element-wise operations need compatible shapes, and 3 elements cannot line up with 2
B. `[11 22 3]` — the shorter array is padded
C. `[11 22]` — the longer array is trimmed
D. `[1 2 3 10 20]` — they are joined

Answer: A

Element-wise operations require the shapes to be compatible; broadcasting can stretch a dimension of size 1, but 3 against 2 has no valid alignment, so it raises. This is not concatenation — `np.concatenate` is what joins arrays.

Q179 *authored*

A tally of exam marks comes out wrong. What is the difference between these two lines?

```
import numpy as np
marks = np.array([[60, 70], [80, 90]])
print(marks.mean(), marks.mean(axis=1))
```

A. `75.0 [65.  85.]` — with no axis the mean is over every element; `axis=1` collapses the columns, giving one mean per row
B. `75.0 [70.  80.]`
C. `[65.  85.]  75.0`
D. `150.0 [65.  85.]`

Answer: A

With no axis the mean is taken over every element: (60+70+80+90)/4 = 75.0. `axis=1` collapses across the columns, giving a mean per row: 65 and 85. Read the axis as "the one that disappears".

Q180 *docs*

A timestamp is needed in the format `2026-08-25`. Which is correct?

```
from datetime import datetime
d = datetime(2026, 8, 25, 14, 30)
```

A. `d.strftime("%Y-%m-%d")`
B. `d.strftime("%y-%M-%D")`
C. `d.format("YYYY-MM-DD")`
D. `str(d.date)`

Answer: A

%Y four-digit year, %m zero-padded month, %d zero-padded day. %y is the two-digit year and %M is minutes, so option B silently produces something plausible and wrong — the worst kind of mistake in a timestamp.

Domain 5 — Debugging and Error Handling (28)

Q101 *docs*

What is the difference between a syntax error and an exception?

A. A syntax error stops the file from compiling at all; an exception is raised while otherwise valid code runs
B. They are two names for the same thing

C. A syntax error can be caught with `try/except`; an exception cannot
D. An exception happens at compile time; a syntax error at run time

Answer: A

A syntax error is found while the source is being parsed, so nothing runs at all and it cannot be caught by a `try` in the same file. An exception is raised by code that parsed fine but hit a problem at run time, and that is what `try/except` is for.

Q102 *docs*

Which exception class sits at the top of the hierarchy, and which should you normally catch?

A. `BaseException` is the root; catch `Exception`, which excludes `SystemExit` and `KeyboardInterrupt`
B. `Exception` is the root; catch `BaseException` for safety
C. `Error` is the root; catch `Error`
D. `RuntimeError` is the root; catch `RuntimeError`

Answer: A

`BaseException` is the root. `Exception` sits below it and deliberately excludes `SystemExit`, `KeyboardInterrupt` and `GeneratorExit`, which is exactly why catching `Exception` is the safe default - it leaves the ways of stopping a program alone.

Q103 *authored*

Which exception does each line raise?

```
import traceback
for src in ["undefined_name", "'a' + 1", "int('abc')", "[1, 2][9]"]:
    try:
        eval(src)
    except Exception as e:
        print(type(e).__name__)
```

A. `NameError` `TypeError` `ValueError` `IndexError`
B. `NameError` `ValueError` `TypeError` `KeyError`
C. `SyntaxError` `TypeError` `ValueError` `IndexError`
D. `NameError` `TypeError` `TypeError` `IndexError`

Answer: A

A name that was never bound gives `NameError`; `'a' + 1` mixes incompatible types so `TypeError`; `int('abc')` gets the right type with an unusable value so `ValueError`; an index past the end gives `IndexError`. Learning to predict the class is most of debugging.

Q104 *docs*

Why does `int("abc")` raise `ValueError` rather than `TypeError`?

A. The type is right - a string is acceptable to `int()` - but the value cannot be interpreted as a number
B. `TypeError` is only for user-defined types
C. It actually raises `TypeError`
D. `ValueError` is raised whenever a conversion function is used

Answer: A

*`TypeError` means the *type* was inappropriate, `ValueError` means the type was fine but the *value* was not. `int()` accepts strings, so `"abc"` is a value problem. `int([])` is a type problem and does raise `TypeError`.*

Q105 *authored*

What does this print?

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("cannot divide by zero")
except Exception:
    print("something else")
```

A. `cannot divide by zero`
B. `something else`

C. both lines
D. it raises an unhandled `ZeroDivisionError`

Answer: A

Handlers are checked in order and the first matching one runs. `ZeroDivisionError` matches first, so the broader handler never fires.

Q106 *docs*

What does this print?

```
try:
    int("x")
except ValueError as e:
    print(type(e).__name__, len(e.args))
```

A. `ValueError 1`
B. `ValueError 0`
C. `Exception 1`
D. it raises a `SyntaxError`

Answer: A

`as e` binds the exception instance. `e.args` is the tuple of arguments it was raised with - here one string - and `str(e)` gives the message.

Q107 *authored*

What does this print?

```
def f():
    try:
        return "try"
    finally:
        print("finally")

print(f())
```

A. `finally` then `try` - the `finally` block runs even when the `try` returns
B. `try` then `finally`
C. `try` only
D. `finally` only

Answer: A

`finally` runs on the way out no matter how the block is left, including a `return`. The return value is computed first, then `finally` runs, then the function actually returns - which is why `finally` prints first.

Q108 *docs*

Why is a bare `except:` discouraged?

A. It catches everything including `KeyboardInterrupt` and `SystemExit`, so it can swallow a user pressing Ctrl-C
B. It is a syntax error in Python 3
C. It catches only `SyntaxError`
D. It re-raises the exception automatically

Answer: A

A bare `except:` catches `BaseException`, so it swallows Ctrl-C and `SystemExit` along with real errors, and makes a program hard to stop or debug. Catch `Exception` if you must be broad, and prefer naming the class.

Q109 *docs*

How do you handle two exception types with one block?

A. `except (ValueError, TypeError):`
B. `except ValueError or TypeError:`
C. `except ValueError, TypeError:`

D. `except [ValueError, TypeError]:`

Answer: A

Multiple types go in a parenthesised tuple. The comma form without parentheses was Python 2 syntax for `as` and is a syntax error now, and `or` evaluates to a single class rather than combining them.

Q110 *authored*

What does this print, and what is wrong with the ordering?

```
try:
    int("x")
except Exception:
    print("general")
except ValueError:
    print("specific")
```

A. `general` - the first matching handler wins, so a broad class listed first makes every later, narrower one unreachable
B. `specific`
C. both
D. it raises a `SyntaxError` for unreachable handlers

Answer: A

Handlers are tried top to bottom and `Exception` matches everything, so listing it first makes every narrower clause below it dead code. Order handlers most specific first - the same rule as `if/elif`.

Q111 *docs*

What does this print?

```
try:
    n = int("7")
except ValueError:
    print("bad input")
else:
    print("parsed", n)
finally:
    print("done")
```

A. `parsed 7` then `done`
B. `bad input` then `done`
C. `parsed 7` only
D. `done` then `parsed 7`

Answer: A

No exception was raised, so `except` is skipped and `else` runs. `finally` runs last, always.

Q112 *docs*

What is the `else` block of a `try` statement for?

A. Code that should run only if the `try` block raised nothing - keeping it out of `try` means its own exceptions are not caught by mistake
B. Code that runs whether or not an exception occurred
C. A fallback that runs when the `except` block fails
D. An alternative spelling of `finally`

Answer: A

*`else` holds the code that should run only on success. Keeping it out of the `try` matters: if it lived inside, an exception raised by *it* would be caught by the same handler, which usually hides a bug.*

Q113 *authored*

What does this print?

```
try:
    raise ValueError("boom")
except ValueError:
    print("except")
else:
    print("else")
finally:
    print("finally")
```

A. `except` then `finally`
B. `except` then `else` then `finally`
C. `else` then `finally`
D. `finally` then `except`

Answer: A

The exception fires, so `except` runs and `else` is skipped entirely. `finally` still runs. `else` means "the try block succeeded", not "otherwise".

Q114 *docs*

How do you define your own exception type?

A. `class InsufficientFunds(Exception): pass`
B. `class InsufficientFunds(Error): pass`
C. `def InsufficientFunds(): raise`
D. `exception InsufficientFunds: pass`

Answer: A

Subclass `Exception` - the documentation's own recommendation. `pass` is enough for a body; the inherited `__init__` already accepts a message. There is no built-in class called `Error`.

Q115 *authored*

What does this print?

```
class TooCold(Exception):
    pass

try:
    raise TooCold("only 3 degrees")
except TooCold as e:
    print(type(e).__name__, e)
```

A. `TooCold only 3 degrees`
B. `Exception only 3 degrees`
C. `TooCold TooCold`
D. it raises a `TypeError` because the class has no `__init__`

Answer: A

The inherited `Exception.__init__` stores the message, and `str(e)` returns it, so printing the instance shows the text. No custom `__init__` is required.

Q116 *docs*

What does this print?

```
class AppError(Exception):
    pass

class DatabaseError(AppError):
    pass
```

```
try:
    raise DatabaseError("timeout")
except AppError:
    print("caught by the base class")
```

A. `caught by the base class` - an `except` clause catches the named class and every subclass of it
B. nothing, because the classes do not match exactly
C. it raises an unhandled `DatabaseError`
D. `timeout`

Answer: A

`except` matches the named class and anything derived from it, so a base exception per application lets one handler cover a whole family. This is why exception hierarchies are worth designing.

Q117 *docs*

Which exception does `10 / 0` raise?

A. `ZeroDivisionError`
B. `ValueError`
C. `ArithmeticError`
D. `FloatingPointError`

Answer: A

Division by zero raises `ZeroDivisionError`. It is a subclass of `ArithmeticError`, so `except ArithmeticError` would also catch it - but the exception actually raised is `ZeroDivisionError`.

Q118 *authored*

Which of these raise `ZeroDivisionError`?

A. `7 // 0`
B. `7 % 0`
C. `0 / 7`
D. `0 ** 0`

Answer: A, B

Floor division and modulo by zero both raise. `0 / 7` is a normal division giving `0.0`, and `0 0` is defined as `1` in Python.

Q119 *authored*

What does `1.0 / 0` do in Python?

A. It raises `ZeroDivisionError` - Python does not return infinity for float division by zero
B. It returns `inf`
C. It returns `nan`
D. It returns `0.0`

Answer: A

Unlike C or JavaScript, Python raises rather than returning infinity. `float('inf')` exists, but you will not get it from dividing by zero.

Q120 *authored*

A report divides a total by a count that is sometimes zero. Which version handles it correctly and still reports a value?

A. Wrap the division in `try`/`except ZeroDivisionError` and set the result to `0` in the handler
B. Test `if count != 0.0:` after the division
C. Catch `ValueError` around the division
D. Use `total // count`, which never raises

Answer: A

Catching the specific exception keeps the happy path clean and handles the one real failure. Option B tests after the division has already raised, and `//` raises just the same.

Q121 *docs*

Which exception does `open("nope.txt")` raise when the file does not exist?

A. `FileNotFoundError`
B. `IOError`
C. `ValueError`
D. `KeyError`

Answer: A

`FileNotFoundError` is the specific subclass raised when the path does not exist. Catching it by name is better than catching `OSError`, which would also swallow permission problems.

Q122 *docs*

Which two statements about `FileNotFoundError` are true?

A. It is a subclass of `OSError`
B. `IOError` is an alias of `OSError`, so `except IOError` also catches it
C. It is a subclass of `ValueError`
D. It is raised when a file exists but is empty

Answer: A, B

`FileNotFoundError` derives from `OSError`, and since Python 3.3 `IOError` is an alias of `OSError` rather than a separate class. An empty file opens fine and raises nothing.

Q123 *authored*

What does this print when `settings.txt` does not exist?

```
try:
    with open("settings.txt") as f:
        data = f.read()
except FileNotFoundError:
    data = "default"
print(data)
```

A. `default`
B. it raises an unhandled `FileNotFoundError`
C. an empty line
D. `settings.txt`

Answer: A

The handler runs, `data` is set to the fallback, and the program continues - the standard shape for an optional config file.

Q124 *authored*

A script must keep running when a config file is missing but must stop loudly when the file exists and is unreadable. What is the right shape?

A. Catch `FileNotFoundError` specifically and let other `OSError` subclasses propagate
B. Use a bare `except:` so nothing can crash the script
C. Catch `Exception` and print a message
D. Check `os.path.exists()` and skip the `try` entirely

Answer: A

Catching `FileNotFoundError` alone handles the case you planned for while letting a permission error or a bad disk surface as a crash you can see. Options B and C hide the second case, and D still leaves a race between the check and the open.

Q181 *authored*

A script silently does nothing and nobody can work out why. What is wrong with the error handling?

```
def load(values):
    try:
        return sum(valeus)
```

```
    except:
        return None

print(load([1, 2, 3]))
```

A. None — the bare `except` swallows a `NameError` caused by the typo `valeus`, hiding a bug that has nothing to do with the data
B. `6` — Python corrects the misspelling
C. it raises a `NameError`
D. `0`

Answer: A

`valeus` is a typo, which raises `NameError` — and the bare `except` catches it along with everything else, converting a crash that would have named the line into a silent `None`. Catch the specific exception you expect, and let the ones you did not expect surface.

Q182 *authored*

What does this return, and why is it a trap?

```
def f():
    try:
        return "try"
    finally:
        return "finally"

print(f())
```

A. `finally` — a `return` inside `finally` replaces the one from the `try`, which silently discards the real result
B. `try`
C. `None`
D. it raises a `SyntaxError`

Answer: A

`finally` always runs, and a `return` inside it overrides the value the `try` was about to return. The real result vanishes with no error. Never `return` from `finally` — use it for cleanup only.

Q183 *authored*

A handler is meant to catch a bad number but never fires. Why?

```
try:
    n = int("twelve")
except TypeError:
    print("caught")
except ValueError:
    print("value")
```

A. `value` — the wrong class was tried first; a string that is not a number raises `ValueError`, not `TypeError`
B. `caught`
C. nothing is printed
D. it raises an unhandled exception

Answer: A

*`int("twelve")` gets an acceptable *type* with an unusable *value*, so it raises `ValueError`. The `TypeError` clause is simply never reached. Predicting the exception class is most of writing a handler that works.*

Q184 *authored*

A custom exception should carry the offending value so the handler can report it. Which works?

```
class BadTemp(Exception):
    pass

try:
    raise BadTemp(-40)
except BadTemp as e:
    print(e.args[0], str(e))
```

A. `-40 -40` — the inherited `__init__` stores whatever it is raised with in `args`
B. it raises a `TypeError` because the class defines no `__init__`
C. `None None`
D. `BadTemp -40`

Answer: A

Subclassing `Exception` with an empty body still inherits an `__init__` that stores its arguments in `args`, and `str(e)` returns the single argument. You only need a custom `__init__` when you want extra structured fields.

Domain 6 — File Handling (21)

Q125 *docs*

Why is the `with` form preferred?

```
with open("notes.txt") as f:
    data = f.read()
```

A. The file is closed automatically when the block ends, even if an exception is raised inside it
B. It reads the file faster
C. It locks the file against other programs
D. It is the only syntax that allows reading

Answer: A

`with` is a context manager: it closes the file on the way out, including when an exception is raised inside the block. Without it a raised exception can leave the handle open and the write unflushed.

Q126 *docs*

What is the default mode of `open(path)`, and what does it mean?

A. `"r"` - open an existing file for reading in text mode
B. `"w"` - open for writing, creating the file if needed
C. `"a"` - open for appending
D. `"rb"` - open for reading in binary mode

Answer: A

The default is `"r"` - read, text mode. Text mode decodes bytes to `str` and translates newlines. `"w"` would destroy the file's contents, which is why the default is read.

Q127 *docs*

What does `open()` return?

A. A file object that you read from, write to and close
B. The contents of the file as a string
C. The path to the file
D. A list of the file's lines

Answer: A

`open()` returns a file object. The contents only arrive when you call `read()`, `readline()` or `readlines()`, or iterate it.

Q128 *docs*

A script opens `"data.txt"` with no folder in front of it. Where does Python look?

A. In the current working directory, which is not necessarily the folder the script lives in
B. Always in the folder containing the script
C. In the Python installation directory
D. In every directory on `sys.path`

Answer: A

A relative path resolves against the current working directory, which is wherever the process was started - not the script's folder. This is why a script works when run from its own directory and fails from anywhere else.

Q129 *authored*

Which two are absolute paths?

A. `C:\data\reports\notes.txt`
B. `/var/reports/notes.txt`
C. `notes.txt`
D. `../notes.txt`

Answer: A, B

An absolute path is complete from the root: a drive letter and backslash on Windows, a leading slash on Linux and macOS. Bare and dotted names are relative.

Q130 *docs*

Which function turns a relative path into an absolute one based on the current working directory?

A. `os.path.abspath(p)`
B. `os.path.realname(p)`
C. `os.fullpath(p)`
D. `os.path.absolute(p)`

Answer: A

`os.path.abspath()` normalises a path against the current working directory. `os.path.realname` does not exist; the real neighbour is `os.path.realpath`, which also resolves symlinks.

Q131 *authored*

What does this print?

```
with open("nums.txt", "w") as f:
    f.write("a\nb\nc\n")

with open("nums.txt") as f:
    print(repr(f.read()))
```

A. `'a\nb\nc\n'` - `read()` returns the whole file as one string, newlines included
B. `['a', 'b', 'c']`
C. `'abc'`
D. `'a'`

Answer: A

`read()` returns the entire file as a single string with newlines intact. `repr()` is used here so the newline characters are visible instead of being printed.

Q132 *docs*

What does this print?

```
with open("nums.txt", "w") as f:
    f.write("a\nb\n")

with open("nums.txt") as f:
    print(f.readlines())
```

A. `['a\n', 'b\n']` - `readlines()` keeps the newline on the end of each line
B. `['a', 'b']`
C. `'a\nb\n'`
D. `['a\nb\n']`

Answer: A

`readlines()` returns a list of lines and keeps the trailing newline on each. Forgetting that is why comparisons against `"a"` fail - the value is `"a\n"`. `strip()` or `splitlines()` removes it.

Q133 *docs*

What does this print?

```
with open("log.txt", "w") as f:
    f.write("one\ntwo\n")

with open("log.txt") as f:
    for line in f:
        print(line.strip())
```

A. one then two
B. `one\ntwo\n`
C. `['one', 'two']`
D. nothing - a file object is not iterable

Answer: A

A file object is its own iterator and yields one line per step, which is the memory-efficient way to read a large file. `strip()` removes the trailing newline before printing.

Q134 *authored*

What does this print, and why does the second read differ?

```
with open("d.txt", "w") as f:
    f.write("hello")

with open("d.txt") as f:
    print(repr(f.read()))
    print(repr(f.read()))
```

A. `'hello'` then `''` - the read position is now at the end of the file
B. `'hello'` then `'hello'`
C. `'hello'` then `None`
D. it raises a `ValueError` on the second read

Answer: A

The file keeps a read position. After the first `read()` it sits at the end, so the second returns an empty string. Call `f.seek(0)` to go back, or store the contents in a variable.

Q135 *docs*

What does this print?

```
with open("out.txt", "w") as f:
    f.write("first")
with open("out.txt", "w") as f:
    f.write("second")
with open("out.txt") as f:
    print(f.read())
```

A. `second` - mode `"w"` truncates the file to empty before writing
B. `firstsecond`
C. `first`
D. `first\nsecond`

Answer: A

Mode `"w"` truncates the file to zero length the moment it is opened, so opening for write and doing nothing still empties it. This is the most destructive default in file handling.

Q136 *docs*

Which mode adds to the end of an existing file without deleting what is already there?

A. `"a"`
B. `"w"`
C. `"r+"` used alone
D. `"x"`

Answer: A

`"a"` positions at the end and preserves what is there. `"w"` truncates, and `"x"` creates but fails if the file already exists.

Q137 *authored*

What does this print?

```
with open("w.txt", "w") as f:
    n = f.write("abc")
    f.write("def")
print(n)
with open("w.txt") as f:
    print(f.read())
```

A. 3 then `abcdef` - `write()` returns the number of characters written and adds no newline
B. 3 then `abc\ndef`
C. 6 then `abcdef`
D. None then `abcdef`

Answer: A

`write()` returns the number of characters written and adds no newline of its own, so two writes run together. Add `"\n"` yourself, or use `print(..., file=f)`.

Q138 *docs*

What is the difference between `os.mkdir()` and `os.makedirs()`?

A. `mkdir` creates one directory and fails if a parent is missing; `makedirs` creates every missing parent along the way
B. `mkdir` is for files and `makedirs` for directories
C. `makedirs` deletes the directory if it already exists
D. They are aliases of each other

Answer: A

`mkdir` creates a single directory and raises `FileNotFoundError` if the parent chain is missing; `makedirs` builds the whole chain. Add `exist_ok=True` to `makedirs` when the directory may already be there.

Q139 *docs*

Which call lists the names of the entries inside a directory?

A. `os.listdir(path)`
B. `os.path.list(path)`
C. `os.dir(path)`
D. `os.readdir(path)`

Answer: A

`os.listdir(path)` returns the entry names - names only, not full paths, which is why joining with `os.path.join` is nearly always the next line. `os.scandir` is the richer modern alternative.

Q140 *docs*

Why use `os.path.join("data", "raw", "a.csv")` instead of `"data" + "/" + "raw" + "/" + "a.csv"`?

A. It inserts the separator the current operating system uses, so the same code works on Windows and on Linux
B. It checks that the file exists
C. It is faster
D. It converts the path to an absolute path

Answer: A

`os.path.join` uses the platform's separator, so the same source runs on Windows and on Linux. It does not touch the filesystem and does not check existence.

Q141 *docs*

What do these print for a path that is an existing file?

```
import os, tempfile
p = os.path.join(tempfile.mkdtemp(), "f.txt")
open(p, "w").close()
print(os.path.exists(p), os.path.isfile(p), os.path.isdir(p))
```

A. `True True False`
B. `True False True`
C. `True True True`
D. `False False False`

Answer: A

`exists` is true for anything at that path, `isfile` narrows it to a regular file and `isdir` to a directory. For an existing file that gives True, True, False.

Q142 *docs*

What does this print?

```
import os
p = os.path.join("reports", "2026", "summary.csv")
print(os.path.basename(p), os.path.dirname(p) != "", os.path.splitext(p)[1])
```

A. `summary.csv True .csv`
B. `summary True .csv`
C. `summary.csv True csv`
D. `reports True .csv`

Answer: A

`basename` returns the last component, `dirname` everything before it, and `splitext` splits into stem and extension with the dot kept on the extension.

Q185 *authored*

A script writes a report, but the file is empty when another program reads it a moment later. What is missing?

```
f = open("report.txt", "w")
f.write("done")
print(open("report.txt").read())
f.close()
print(open("report.txt").read())
```

A. an empty line then `done` — the text sits in a buffer until the file is closed, which is exactly what `with` guarantees
B. `done` then `done`
C. it raises a `ValueError`
D. an empty line twice

Answer: A

Writes are buffered and are not guaranteed to reach disk until the file is closed or flushed, so the first read sees an empty file. `with open(...) as f:` closes on the way out including on exception, which is why it is the default advice.

Q186 *authored*

A script works from its own folder and fails from anywhere else. Which fix is correct?

A. Build the path from the script's own location, for example `os.path.join(os.path.dirname(os.path.abspath(__fi "data.txt")`
B. Always call `os.chdir()` to the user's home directory first

C. Use `open("./data.txt")`, which is always relative to the script
D. Move the file to the Python installation directory

Answer: A

A relative path resolves against the current working directory, which is wherever the process was launched — not the script's folder. `__file__` gives the script's own location, and building the path from it makes the script runnable from anywhere. `./data.txt` is still relative to the working directory, so option C changes nothing.

Q187 *docs*

A script lists a folder and then opens each file, and it fails with `FileNotFoundError` unless it is run from inside that folder. Why?

```
import os, tempfile
d = tempfile.mkdtemp()
open(os.path.join(d, "a.txt"), "w").close()
print(os.listdir(d))
```

A. `os.listdir()` returns bare names, not paths — rejoin each one with `os.path.join(d, name)` before opening it
B. `os.listdir()` returns absolute paths and the script is corrupting them
C. `os.listdir()` cannot see files created in the same script
D. The folder must be added to `sys.path`

Answer: A

`os.listdir()` returns bare entry names. Opening one only works while the working directory happens to be that folder. Rejoin with `os.path.join(directory, name)` — or use `os.scandir()`, whose entries carry a usable `.path`.

Domain 7 — GUI Programming (18)

Q143 *docs*

How do you get Tkinter?

A. It ships with the standard CPython installation - `import tkinter` works with no install step
B. `pip install tkinter`
C. `pip install tk`
D. It must be downloaded from the Tcl website

Answer: A

Tkinter is part of the standard library on a normal CPython install, so no `pip install` is needed - and `pip install tkinter` fails, which confuses beginners. Some Linux distributions package it separately as `python3-tk`.

Q144 *authored*

What distinguishes a GUI program from a command-line program?

A. The user acts on visible widgets and the program responds to events, rather than reading a fixed sequence of input
B. A GUI program cannot read or write files
C. A GUI program runs faster
D. A GUI program needs no main loop

Answer: A

A GUI presents widgets and reacts to events in whatever order the user produces them. A command-line program follows its own fixed sequence. That inversion of control is the core idea of the domain.

Q145 *docs*

What does `root.mainloop()` do?

A. It starts the event loop, which waits for user actions and dispatches them to handlers - the call blocks until the window closes
B. It draws the window once and returns immediately
C. It repeatedly redraws the window sixty times a second

D. It is optional decoration

Answer: A

`mainloop()` starts the event loop: it waits for events and dispatches them to the bound handlers, and it does not return until the window is closed. Nothing appears without it.

Q146 *docs*

Match the widget to its job. Which two pairings are correct?

A. `Label` displays text or an image the user cannot edit
B. `Entry` accepts a single line of typed text
C. `Button` displays scrollable multi-line text
D. `Frame` accepts numeric input only

Answer: A, B

`Label` shows non-editable text or an image, `Entry` takes one line of typed input. `Button` triggers a command, and `Frame` is an invisible container used to group widgets. Multi-line text is the `Text` widget.

Q147 *docs*

Which call reads what the user typed into an `Entry` widget named `box`?

A. `box.get()`
B. `box.text`
C. `box.value()`
D. `box.read()`

Answer: A

`Entry.get()` returns the current contents as a string. There is no `.text` attribute holding the value; `insert()` and `delete()` are the write side.

Q148 *docs*

Which widget holds multi-line, editable text?

A. `Text`
B. `Label`
C. `Entry`
D. `Message`

Answer: A

`Text` is the multi-line editable widget. `Label` and `Message` display text without editing, and `Entry` is single-line.

Q149 *authored*

What does this pass to the button, and what is the bug?

```
import tkinter as tk

def say_hi():
    print("hi")

root = tk.Tk()
b = tk.Button(root, command=say_hi())
```

A. `say_hi()` calls the function immediately and passes its return value None; the button gets no callback. Pass `command=say_hi` with no parentheses
B. It works correctly - the parentheses are required
C. It raises a `TypeError` at the `Button` call
D. The button will call `say_hi` twice

Answer: A

`command=say_hi()` runs the function right there and hands the button its return value, so the message prints once at startup and the button then does nothing. Pass the function object itself: `command=say_hi`. Use `lambda: say_hi(arg)` when you need arguments.

Q150 *docs*

Which method attaches a handler to an event such as a key press or a mouse click?

A. `widget.bind("<Button-1>", handler)`
B. `widget.on("click", handler)`
C. `widget.listen("<Button-1>", handler)`
D. `widget.attach("<Button-1>", handler)`

Answer: A

`bind(sequence, handler)` connects an event to a callback, with sequences such as `<Button-1>` for a left click and `<Return>` for the Enter key. The handler receives an event object.

Q151 *authored*

What does "event-driven programming" mean?

A. The program sets up handlers and then waits; the order in which code runs is decided by what the user does
B. The program runs its statements top to bottom and exits
C. Every function is called on a timer
D. Events are processed only when the program is idle

Answer: A

In an event-driven program you register handlers and hand control to the event loop; the user decides what runs and when. That is why a GUI has no single top-to-bottom path through the code.

Q152 *docs*

Which three geometry managers does Tkinter provide?

A. `pack`, `grid` and `place`
B. `pack`, `align` and `float`
C. `layout`, `grid` and `anchor`
D. `row`, `column` and `cell`

Answer: A

Tkinter has exactly three geometry managers: `pack` stacks against a side, `grid` uses rows and columns, and `place` uses explicit coordinates.

Q153 *authored*

What goes wrong when you call `pack()` on one widget and `grid()` on another inside the same parent container?

A. Tkinter raises an error - a single container may be managed by only one geometry manager
B. Both are honoured and the widgets overlap
C. The second call is silently ignored
D. Nothing - mixing them is the recommended approach

Answer: A

A container may be managed by one geometry manager only; mixing `pack` and `grid` in the same parent raises `TclError`. You can still use different managers in different frames, which is the normal way to build a mixed layout.

Q154 *docs*

In `widget.grid(row=1, column=2)`, what do the numbers mean?

A. The row and column of the cell the widget occupies in its parent's grid, counted from 0
B. The pixel offset from the top-left of the window
C. The width and height of the widget
D. The stacking order of the widget

Answer: A

`row` and `column` are the grid cell coordinates within the parent, numbered from zero. `sticky`, `rowspan` and `columnspan` refine the placement.

Q155 *docs*

Which two statements about `pack()` are true?

A. It places widgets in order against one side of the container
B. `side` accepts values such as `"top"`, `"left"`, `"right"` and `"bottom"`
C. It positions widgets at exact pixel coordinates
D. It requires `row` and `column` arguments

Answer: A, B

`pack` places each widget against a side of the remaining space, controlled by `side` with `top`, `left`, `right` or `bottom`. Exact coordinates are `place`, and `row`/`column` belong to `grid`.

Q156 *docs*

What is `root = tk.Tk()`?

A. The main application window, created once and used as the parent for the other widgets
B. A dialog box
C. The event loop itself
D. A layout manager

Answer: A

`Tk()` creates the main window. Every other widget takes it - or a frame inside it - as its parent, and `mainloop()` is called on it. Extra windows use `Toplevel()`.

Q157 *authored*

A beginner creates a `Label` and calls `mainloop()`, but the window is empty. What is missing?

```
import tkinter as tk
root = tk.Tk()
lbl = tk.Label(root, text="hello")
root.mainloop()
```

A. The label was never handed to a geometry manager - it needs `lbl.pack()`, `lbl.grid()` or `lbl.place()`
B. `Label` needs a `visible=True` argument
C. `mainloop()` must be called on the label
D. The text argument must be a variable

Answer: A

Creating a widget does not display it. Until a geometry manager is called it has no place in the layout and nothing is drawn, so the window opens empty with no error. This is the most common first Tkinter bug.

Q188 *authored*

A form reads the entry box the moment the window is built and always gets an empty string. Why?

A. The read happens before the event loop starts, so the user has not typed anything yet — read the value inside the button's callback instead
B. `Entry.get()` only works after `destroy()`
C. `Entry` needs `readable=True`
D. The entry must be packed twice

Answer: A

*Widget code runs to completion *before* `mainloop()` starts accepting input, so a read at build time happens before the user has typed. Read the value inside the callback, which runs in response to the click. This is the practical consequence of event-driven control flow.*

Q189 *authored*

A window is meant to show a label and a button side by side in a row, then a status bar underneath spanning both. Which manager fits without fighting?

A. `grid`, placing the two widgets at row 0 columns 0 and 1 and the status bar at row 1 with `columnspan=2`
B. `place`, with pixel coordinates for all three
C. `pack` with `side="left"` for all three

D. Mixing `grid` for the row and `pack` for the status bar in the same container

Answer: A

`grid` is built for rows and columns, and `columnspan=2` is exactly how one widget stretches across both. `pack` with `side="left"` would put the status bar beside them, not under. Option D is the error from Q153 — one container, one geometry manager.

Q190 *authored*

A button is built inside a function and the window shows it, but clicking it does nothing and no error appears. Which is the most likely cause?

A. `command` was given the *result* of calling the handler rather than the handler itself, so the button holds `None`
B. Buttons cannot be created inside a function
C. `mainloop()` must be called on the button
D. The button needs `clickable=True`

Answer: A

`command=handler()` runs the function immediately and hands the button its return value, usually `None` — so the button is wired to nothing and clicking is silent. Pass the function object: `command=handler`, or `command=lambda: handler(arg)` when arguments are needed. Silent failure is what makes it hard to spot.

Part II — Practice-exam set (133)

Domain 1 — Introduction and Setup (37)

1.

What will the value of the i variable be when the following loop finishes its execution?

```
for i in range(10):
    pass
```

A. 10
B. the variable becomes unavailable
C. 11
D. 9

Answer: D

*A `for` loop leaves its control variable holding the **last value it took**, and `range(10)` stops at 9. The variable is still in scope after the loop - Python has no block scope - which is why B is wrong.*

2.

The following expression 1+-2 is:

A. equal to 1
B. invalid
C. equal to 2
D. equal to -1

Answer: D

`1+−2` parses as `1 + (−2)`. The second − is a unary minus attached to the 2, not a second operator, so this is perfectly valid and gives -1.

3.

A compiler is a program designed to

A. rearrange the source code to make it clearer
B. check the source code in order to see if it's correct
C. execute the source code
D. translate the source code into machine code

Answer: B, D

A compiler checks the source for correctness and translates it to machine code. It does not execute the result - that is the job of the program it produced - and it does not reformat your source.

4.

What is the output of the following piece of code?

```
a='ant'
b='bat'
c='camel'
print(a,b,c,sep='')
```

A. ant' bat' camel
B. ant'bat'camel
C. antbatcamel
D. SyntaxError

Answer: C

*`sep` controls what `print` puts *between* its arguments, and here it is set to an empty string, so the three words run together as `antbatcamel`. The default `sep` is a single space.*

5.

What is the expected output of the following snippet?

```
i=5
while i>0:
    i=i//2
    if i%2==0:
        break
    else:
        i+=1
print(i)
```

A. the code is erroneous
B. 3
C. 7
D. 15

Answer: A

This question is broken. *The code runs fine and prints **2**, but none of the four options offers 2 - and option A claims the code is erroneous, which it is not. `i=5` becomes `5//2 = 2`, then `2 % 2 == 0` is true so `break` fires immediately.*

*Traced by running it: `i = 5 // 2` is 2, and `2 % 2 == 0` is true, so `break` fires on the first pass and `print(i)` shows **2**.*

6.

How many lines does the following snippet output?

```
for i in range(1,3):
    print('**',end='')
else:
    print('**')
```

A. three
B. one
C. two
D. four

Answer: B

This question's key was wrong and is corrected here. *It asks how many *lines* are printed. `print('**', end='')` runs twice and emits no newline, then the loop's `else` runs `print('**')` which does. The whole output is `******` followed by one newline - **one line**, not two.*

`print('**', end='')` suppresses the newline, so the two loop passes emit `****` on one line. A `while/for else` block runs when the loop finishes without `break`, adding `**` and a newline. One line of six stars.

7.

Which of the following literals reflect the value given as 34.23?

A. 3.42E+01
B. 3.42E+01
C. 3.42E-03
D. 3.42E+05

Answer: A, B

This question is broken. *Options A and B are byte-identical (`3.42E+01`) and both are keyed correct, so one of them is a typo for something else. On the content: 34.23 is 3.423E+01, so `3.42E+01` is the intended form.*

34.23 in scientific notation is 3.423 x 10^1, written `3.423E+01`. The exponent is the number of places the point moves.

8.

Select the valid fun() invocations:

```
def fun(a,b=0):
    return a*b
```

A. fun(b=1)

B. fun(a=0)
C. fun(b=1,0)
D. fun(1)

Answer: B, D

b has a default so it may be omitted, but a has none and must be supplied. `fun(b=1)` leaves a unset. `fun(b=1, 0)` is a syntax error - a positional argument can never follow a keyword one.

9.

What is the expected output of the following code?

```
def f(n):
    if n==1:
        return '1'
    return str(n)+f(n-1)
print(f(2))
```

A. 21
B. 12
C. 3
D. 1

Answer: A

`f(2)` returns `str(2) + f(1)`, and `f(1)` returns `'1'`. String concatenation, not addition, so the result is the text `21`.

10.

What is the expected behaviour of the following snippet?

```
def x():
    return 2
x=1+x()
print(x)
```

A. cause a runtime exception on line 02
B. cause a runtime exception on line 01
C. cause a runtime exception on line 03
D. print 3

Answer: D

The `def` runs first and binds `x` to the function; then `x = 1 + x()` calls it while the name still refers to the function, gets 2, adds 1, and rebinds `x` to 3. Rebinding a name is always legal in Python.

11.

What is the expected behaviour of the following code?

```
def f(n):
    for i in range(1,n+1):
        yield i
print(f(2))
```

A. print 4321
B. print <generator object f at (some hex digits)>
C. cause a runtime exception
D. print 1234

Answer: B

*A function containing `yield` is a **generator function**. Calling it does not run the body - it returns a generator object, which is what gets printed. You would need `list(f(2))` or a `for` loop to see the values.*

12.

If you need a function that does nothing, what would you use instead of XXX?

```
def idler(): XXX
```

A. pass
B. return
C. exit
D. None

Answer: A, B

Both `pass` and `return` are valid single-statement bodies. `pass` is the explicit do-nothing statement; a bare `return` exits immediately, also returning None. `exit` would end the program and None is a value, not a statement.

13.

Which of the following words can be used as a variable name?

A. for
B. TRUE
C. TRUE
D. For

Answer: C, D

***Options B and C are identical** (TRUE), so one is a typo for something else. The answer still holds: `for` is a reserved keyword and cannot be a name, while TRUE and `For` are ordinary identifiers - Python's boolean is spelled `True`, so TRUE is just a name, and names are case-sensitive.*

`for` is a reserved keyword, so it cannot be a name. `For` is a different identifier - names are case-sensitive - and TRUE is an ordinary name too, since Python's boolean is spelled `True`.

14.

Python strings can be 'glued' together using the operator:

A. &
B. *
C. ",
D. +

Answer: D

*+ concatenates strings. * repeats one (`'ab' * 3`), and & is bitwise AND, which strings do not support.*

15.

A keyword

A. can be used as an identifier
B. is defined by Python's lexis
C. is also known as a reserved word
D. cannot be used in the user's code

Answer: B, C

A keyword is part of the language's own lexis - a reserved word - so it cannot be used as an identifier. It very much does appear in your code; that is the point of `if`, `for` and `def`.

16.

How many stars (*) does the snippet print?

```
S='**********'
print(s)
```

A. the code is erroneous
B. five
C. four

D. None (11 stars)

Answer: A

Python is case-sensitive, so S and s are different names. S was assigned, s never was, and `print(s)` raises `NameError`.

17.

Assuming that V holds integer 2, which operator should be used instead of OPER to make V OPER 1 equal to 1?

A. «<
B. »>
C. »
D. «

Answer: C

» shifts bits right, and shifting binary 10 (decimal 2) right by one gives binary 1. There is no »> operator in Python.

18.

How many stars (*) does the following snippet print?

```
i=3
while i>0:
    i-=1
    print('*')
else:
    print('*')
```

A. the code is erroneous
B. five
C. three
D. four

Answer: D

The loop runs three times, printing a star each pass, and the `else` clause runs once when the loop ends without `break` - four stars in total. Verified by running it.

19.

UNICODE is:

A. the name of an operating system
B. a standard for encoding and handling texts
C. the name of a programming language
D. the name of a text processor

Answer: B

Unicode is a standard that assigns a number, called a code point, to every character in every writing system, plus encodings such as UTF-8 that store those numbers as bytes.

20.

Which of the equations are True?

A. chr(ord(x)) == x
B. ord(ord(x)) == x
C. chr(chr(x)) == x
D. ord(chr(x)) == x

Answer: A, D

`ord()` turns a character into its code point and `chr()` turns a code point back into a character, so each undoes the other. The other two combinations pass the wrong type in.

21.

A two-parameter lambda function raising its first parameter to the power of the second parameter should be declared as:

A. lambda (x,y) = x**y
B. lambda (x,y): x**y
C. def lambda (x,y): return x**y
D. lambda x, y: x**y

Answer: D

A lambda is written `lambda params: expression` - no parentheses around the parameters, no `return`, no `def`. The expression's value is what it returns.

22.

What is the expected output of the following code?

```
def f(n):
    if n==1:
        return 1
    return n+f(n-1)
print(f(2))
```

A. 21
B. 12
C. 3
D. none

Answer: C

*`f(2)` returns `2 + f(1)`, and `f(1)` hits the base case and returns 1. These are numbers, so it adds to **3** - compare with the near-identical question that concatenates strings and gives `21`.*

23.

Which function or operator should you use to obtain the answer True or False to the question: 'Do two variables refer to the same object?'

A. The = operator
B. The isinstanceOf function
C. The id() function
D. The is operator

Answer: D

`is` compares identity - whether two names point at the same object. `==` compares value, and `=` assigns. `id()` gives you the identity number but you would have to compare the two results yourself.

24.

Select the true statements related to PEP 8 programming recommendations.

A. You should use the not ... is operator rather than the is not operator
B. You should make object type comparisons using the isinstance() method
C. You should write code in a way that favours the CPython implementation over PyPy, Cython, and Jython
D. You should not write string literals that rely on significant trailing whitespace

Answer: B, D

PEP 8 says to use `isinstance()` for type comparisons and warns against trailing whitespace in literals, which is invisible and easily lost. It prefers `is not` over `not ... is`, and explicitly tells you not to favour one implementation.

25.

Which of the following statements are true?

A. ASCII stands for Information Interchange
B. a code point is a number assigned to a given character
C. ASCII is synonymous with UTF-8

D. \e is an escape sequence used to mark the end of lines

Answer: A, B

*A code point is the number assigned to a character. ASCII is *American Standard Code for Information Interchange* - the option here drops the first half. UTF-8 is a Unicode encoding, not a synonym for ASCII, and `\n` marks a line end, not `\e`.*

26.

What is the expected output of the following code?

```
myTu=('a','b','c')
m=tuple(map(lambda x: chr(ord(x)+1), myTu))
print(m[-2])
```

A. a
B. c
C. an exception is raised
D. b

Answer: B

***This question's key was wrong and is corrected here.** The bank keys b; running it gives c.*

*The bank keys **b**. Running it: `('a','b','c')` maps through `chr(ord(x)+1)` to `('b','c','d')`, so `m[-2]` - the second from the end - is **c**.*

27.

Assuming that the following code has been executed successfully, which of the expressions evaluate to True?

```
def f(x,y):
    nom,denom=x,y
    def g():
        return nom/denom
    return g
a=f(1,2)
b=f(3,4)
```

A. b()==4
B. a!=b
C. a is not None
D. a()==4

Answer: B, C

`f` returns a new function each call, so `a` and `b` are different objects and neither is `None`. Each closure remembers its own `nom` and `denom`, so `a()` is 0.5 and `b()` is 0.75.

28.

What is the expected behaviour of this code?

```
def foo(x,y):
    return y(x)+(x+1)
print(foo(1, lambda x: x*x))
```

A. 3
B. 5
C. 4
D. an exception is raised

Answer: A

*`y(x)` is `1*1 = 1`, then `+ (x+1)` adds 2, giving **3**. Note the second term is not passed through the lambda - compare the near-identical question where it is.*

29.

Which of the following lambda definitions are correct?

A. lambda x,y: (x,y)
B. lambda x,y: return x/y - x%y
C. lambda x,y: x/y - x%y
D. lambda x,y = x/y - x%y

Answer: A, C

A lambda's body is a single expression, so there is no `return` and no =. Returning a tuple is fine because `(x, y)` is an expression.

30.

What is the expected behaviour of this code?

```
x = 3 % 1
y = 1 if x > 0 else 0
print(y)
```

A. the code is erroneous and it will not execute
B. it outputs 1
C. it outputs -1
D. it outputs 0

Answer: D

`3 % 1` is 0 - one divides into three exactly, leaving no remainder. So `x > 0` is false and the conditional expression yields 0.

31.

What is the expected behaviour of this code?

```
x = 8 ** (1/3)
y = 2. if x < 2.3 else 3.
print(y)
```

A. the code is erroneous and it will not execute
B. it outputs 2.0
C. it outputs 2.5
D. it outputs 3.0

Answer: B

`8 ** (1/3)` is the cube root of 8, which is 2.0. `2.0 < 2.3` is true, so the conditional expression yields `2.` - and `2.` is a float literal, so it prints as `2.0`.

32.

Which of the following statements are true?

A. an escape sequence can be recognised by the / sign put in front of it
B. ASCII stands for Internal Information
C. ASCII is a subset of UNICODE
D. a code point is a number assigned to a given character

Answer: C, D

ASCII's 128 characters are the first 128 code points of Unicode, so it is a subset, and a code point is the number assigned to a character. An escape sequence starts with a backslash, not a slash.

33.

What is the expected behaviour of this code?

```
string='123'
dummy=0
for character in reversed(string):
    dummy += int(character)
print(dummy)
```

A. it outputs 321
B. it outputs 123
C. it outputs 6
D. it raises an exception

Answer: C

`reversed()` walks the characters backwards, but addition does not care about order - 3 + 2 + 1 is the same 6. The reversal is a red herring.

34.

Which of the following expressions evaluate to True?

A. ord('2') - ord('2') == ord('0')
B. len('6E6E6E6E') > 0
C. chr(ord('a')+1) == 'B'
D. len('') == 1

Answer: A, B

This question is broken. *It asks for two, but only B is true. Option A - `ord('2') - ord('2') == ord('0')` - is 0 == 48, which is* ***false****, and it is keyed correct. `chr(ord('a')+1)` is `'b'`, not `'B'`, and an empty string has length 0.*

`ord('2') - ord('2')` is 0, and `ord('0')` is 48 - so A is false as written and this key is worth checking. B is true: an eight-character string has a length above 0. `chr(ord('a')+1)` is `'b'`, not `'B'`, and an empty string has length 0.

35.

What is the expected behaviour of this code?

```
def foo(x,y):
    return y(x)+y(x+1)
print(foo(1, lambda x: x*x))
```

A. 4
B. 3
C. an exception is raised
D. 5

Answer: D

`y(x)` is `1*1 = 1` and `y(x+1)` is `2*2 = 4`, so the sum is ***5****. Both terms go through the lambda here - compare the near-identical question where the second does not.*

36.

Which of the following expressions evaluate to True?

A. ord('2') - ord('2') == ord('0')
B. chr(ord('A')+1) == 'B'
C. len('') == 1
D. len('0') == ('E'*'E')

Answer: A, B

This question is broken. *Same defective option as the other `ord` question: `ord('2') - ord('2') == ord('0')` is 0 == 48, which is* ***false****, and it is keyed correct. Only B is true.*

`chr(ord('A')+1)` is `'B'` - `ord` gives the code point, +1 moves one letter on, `chr` converts back. `'E'*'E'` would raise `TypeError`, since you cannot multiply two strings.

37.

Which of the following is not a version of Python?

A. Python 2
B. Python 3
C. Python X
D. Python 4

Answer: C

There is no Python X. The two versions that matter are Python 2, now end-of-life, and Python 3, which is what you should be learning.

Domain 2 — Data Structures (20)

38.

Assuming that the following snippet has been successfully executed, which of the equations are True?

```
a=[1]
b=a
a[0]=0
```

A. len(a)==len(b)
B. b[0]+1==a[0]
C. a[0]==b[0]
D. a[0]+1==b[0]

Answer: A, C

`b = a` does not copy - both names point at the same list. Changing `a[0]` changes what `b[0]` shows, so the two are always equal and always the same length.

39.

Assuming that the following snippet has been successfully executed, which of the equations are False?

```
a=[0]
b=a[:]
a[0]=1
```

A. len(a)==len(b)
B. a[0]-1==b[0]
C. a[0]==b[0]
D. b[0]-1==a[0]

Answer: C, D

*`b = a[:]` **does** copy, so the two lists are now independent. Setting `a[0] = 1` leaves `b[0]` at 0, which makes `a[0] == b[0]` false and `b[0] - 1 == a[0]` false (that would be -1 == 1).*

40.

Which of the following statements are false?

A. Python strings are actually lists
B. Python strings can be concatenated
C. Python strings can be sliced like lists
D. Python strings are mutable

Answer: A, D

*Strings are their own type, not lists, and they are **immutable** - you cannot assign to `s[0]`. They can be concatenated and sliced, which is what makes them feel list-like.*

41.

Which of the following sentences are true?

A. Lists may not be stored inside tuples
B. Tuples may be stored inside lists
C. Tuples may not be stored inside tuples
D. Lists may be stored inside lists

Answer: B, D

*Lists and tuples can each hold the other, and can nest inside themselves. The only real restriction is the other way round: a *list* cannot be a dictionary key or a set member, because it is mutable and therefore unhashable.*

42.

Assuming that String is six or more letters long, the following slice string[1:-2] is shorter than the original string by:

A. four chars
B. three chars
C. one char
D. two chars

Answer: B

`[1:-2]` drops one character from the front and two from the end - three in total. Negative indices count back from the end, and the stop is exclusive.

43.

What is the expected output of the following snippet?

```
lst=[1,2,3,4]
lst=lst[-3:-2]
lst=lst[-1]
print(lst)
```

A. 1
B. 4
C. 2
D. 3

Answer: C

Run it: `lst[-3:-2]` takes one element, the third from the end, giving `[2]`. Then `lst[-1]` on that one-element list is the element itself, 2.

44.

How many elements will the list2 list contain after execution of the following snippet?

```
list1=[False for i in range(1,10)]
list2=list1[-1:1:-1]
```

A. zero
B. five
C. seven
D. three

Answer: C

*`list1` has 9 elements, indices 0 to 8. `[-1:1:-1]` starts at index 8 and steps backwards, stopping *before* index 1, so it takes 8, 7, 6, 5, 4, 3, 2 - **seven**.*

45.

What would you use instead of XXX if you want to check whether a certain key exists in a dictionary called dict?

A. 'key' in dict
B. dict['key'] != None
C. dict.exists('key')
D. 'key' in dict.keys()

Answer: A, D

*`in` on a dictionary tests its **keys**, so `'key' in dict` is the idiomatic form; `'key' in dict.keys()` does the same thing more slowly. There is no `.exists()`, and comparing to None fails when the stored value genuinely is None.*

46.

You need data which can act as a simple telephone directory. You can obtain it with the following clauses

A. dir={'Mom':5551234567,'Dad':5557654321}
B. dir={'Mom':'5551234567','Dad':'5557654321'}

C. dir={Mom:5551234567,Dad:5557654321}
D. dir={Mom:'5551234567',Dad:'5557654321'}

Answer: A, B

*Dictionary **keys** must be written as values - here quoted strings. Without the quotes, `Mom` is a name Python tries to look up, and raises `NameError`. Both keyed options quote the keys; whether the numbers are strings or ints is free choice.*

47.

What is the expected output of the following code?

```
str='abcdef'
def fun(s):
    del s[2]
    return s
print(fun(str))
```

A. abcef
B. The program will cause a runtime exception/error
C. acdef
D. abdef

Answer: B

Strings are immutable, so `del s[2]` raises `TypeError: 'str' object doesn't support item deletion`. To remove a character, build a new string: `s[:2] + s[3:]`.

48.

Which of the listed actions can be applied to the following tuple?

```
tup=()
```

A. tup[0]
B. tup.append(0)
C. tup[0]
D. del tup

Answer: C, D

This question is broken. *Options A and C are identical (`tup[0]`), and one of them is keyed correct. On an *empty* tuple `tup[0]` raises `IndexError`, so it cannot be applied at all; `del tup` genuinely can.*

A tuple is immutable, so there is no `append` and no item assignment. `del tup` deletes the whole name, which is always allowed.

49.

Executing the snippet dct={'pi':3.14}; dct['pi']=3.1415 will cause the dct:

A. to hold two keys named 'pi' linked to 3.14 and 3.1415 respectively
B. to hold two key named 'pi' linked to 3.14 and 3.1415
C. to hold one key named 'pi' linked to 3.1415
D. to hold two keys named 'pi' linked to 3.1415

Answer: C

A dictionary key is unique. Assigning to an existing key replaces its value rather than adding a second entry, so the dictionary still has one key.

50.

How many elements will the list1 list contain after execution of the following snippet?

```
list1 = 'don't think twice, do it!'.split(',')
```

A. two
B. zero
C. one
D. three

Answer: A

`split(',')` cuts at each comma. The text has one comma, so it produces two pieces. Splitting always yields one more piece than the number of separators found.

51.

If you want to transform a string into a list of words, what invocation would you use? (Expected output: The, Catcher, in, the Rye,)

```
S='The Catcher in the Rye'
L=# put proper invocation
```

A. s.split()
B. split(s,' ')
C. s.split(' ')
D. split(s)

Answer: A

*`split()` with no argument splits on any run of whitespace and discards empty pieces, which is what you want for words. It is a *method* on the string, so `s.split()` - there is no bare `split()` function.*

52.

Assuming that lst is a four-element list, is there any difference between these two statements? del lst del lst[:]

A. yes, the first line empties the list, the second deletes the list as a whole
B. yes, the first line deletes the list as a whole, the second just empties the list
C. no, there is no difference
D. yes, the first line deletes the list as a whole, the second removes all elements except the first

Answer: B

`del lst` removes the name entirely, so using it afterwards raises `NameError`. `del lst[:]` deletes every element through a slice, leaving an empty list still bound to the name.

53.

Which of the following expressions evaluate to True?

A. str(1-1) in '123456789'
B. 'dcb' not in 'abcde'[::-1]
C. 'phd' in 'alpha'
D. 'True' not in 'False'

Answer: A, D

This question is broken. *It asks for two true expressions, but only D is true. `str(1-1)` is `'0'`, and `'0'` does not appear in `'123456789'`, so A is false - yet A is keyed. `'abcde'[::-1]` is `'edcba'`, which does contain `'dcb'`, and `'phd'` is not in `'alpha'`.*

`str(1-1)` is `'0'`, which is not in `'123456789'` - so A is false as written and this key is worth checking. `'True' not in 'False'` is true: it is a substring test, and `'True'` does not appear inside `'False'`.

54.

What is the expected behaviour of the following code?

```
the_list='alpha;beta;gamma'.split(':')
the_string='.'.join(the_list)
print(the_string.isalpha())
```

A. it raises an exception
B. it outputs True
C. it outputs False
D. it outputs nothing

Answer: C

`.split(':')` finds no colon, so it returns the whole string as a single-element list. Joining that gives the original text back, and `isalpha()` is False because of the semicolons.

55.

Assuming that the snippet below has been executed successfully, which of the following expressions evaluate to True?

```
string='python'[::2]
string=string[-1]+string[-2]
```

A. string[0]=='o'
B. string is None
C. len(string)==3
D. string[0]==string[-1]

Answer: A

The stem says choose two but the bank keys only one. A is correct: `'python'[::2]` is `'pto'`, then `string[-1] + string[-2]` is `'ot'`, so `string[0]` is `'o'`.

`'python'[::2]` takes every second character - `'pto'`. Then `string[-1] + string[-2]` is `'o'` reversed onto `'t'`, giving `'ot'`, so `string[0]` is `'o'` and the length is 2.

56.

Which of the following expressions evaluate to True?

A. 'in' in 'Thames'
B. 'in' in 'in'
C. 'in not' in 'not'
D. 't'.upper() in 'Thames'

Answer: B, D

`in` is a substring test. `'Thames'` does not contain `'in'`; `'in'` trivially contains itself; and `'t'.upper()` is `'T'`, which `'Thames'` does start with.

57.

Which of the following invocations are valid?

A. sort('python')
B. 'python'.find('e')
C. 'python'.index('eth')
D. 'python'.sorted()

Answer: B, C

`find()` and `index()` are both real string methods, so both calls are valid syntax - `find` returns -1 when absent and `index` raises. There is no bare `sort()` function for strings and no `.sorted()` method; the built-in is `sorted('python')`.

Domain 3 — Object-Oriented Programming (32)

58.

Is it possible to safely check if a class/object has a certain attribute?

A. yes, by using the hashtart attribute
B. yes, by using the hashtart() method
C. yes, by using the hasattr() function
D. no, it is not possible

Answer: C

`hasattr(obj, 'name')` returns True or False without raising, which is what makes it the safe check. The alternative is `getattr(obj, 'name', default)`.

59.

The first parameter of each method:

A. holds a reference to the currently processed object
B. is always set to None

C. is set to a unique random value
D. is set by the first argument's value

Answer: A

The first parameter receives the instance the method was called on, conventionally named `self`. `obj.method()` is shorthand for `Class.method(obj)`.

60.

The simplest possible class definition in Python can be expressed as:

A. class X:
B. class X: pass
C. class X: return
D. class X: {}

Answer: B

A class body cannot be empty, so it needs at least one statement - `pass` is the null statement that satisfies that. `class X:` alone is a syntax error.

61.

A variable stored separately in every object is called:

A. there are no such variables, all variables are shared among objects
B. a class variable
C. an object variable
D. an instance variable

Answer: D

An instance variable is stored on each object separately, normally assigned as `self.x = ...` inside `__init__`. A class variable lives on the class and is shared by every instance.

62.

The following class hierarchy is given. What is the expected output of the code?

```
class A:
    def a(self):
        print('A',end=' ')
    def b(self):
        self.a()
...
B().do()
C().do()
```

A. BB
B. CC
C. AA
D. BC

Answer: D

*`b()` finds b on the base class, which calls `self.a()`. Because `self` is the *subclass* instance, the subclass's own a runs - so each class prints its own letter. That is polymorphism through the method resolution order.*

63.

If any of a class's components has a name that starts with two underscores (__), then:

A. the class component's name will be mangled
B. the class component has to be an instance variable
C. the class component has to be a class variable
D. the class component has to be a method

Answer: A

*Two leading underscores trigger **name mangling** - `__x` inside `class C` becomes `_C__x`. It prevents accidental clashes in subclasses; it is not access control, and the attribute is still reachable under the mangled name.*

64.

A function called issubclass(c1,c2) is able to check if:

A. c1 and c2 are both subclasses of the same superclass
B. c2 is a subclass of c1
C. c1 is a subclass of c2
D. c1 and c2 are not subclasses of the same superclass

Answer: C

`issubclass(c1, c2)` reads in argument order: is the first a subclass of the second. `isinstance` follows the same order with an object first.

65.

A class constructor

A. can return a value
B. cannot be invoked directly from inside the class
C. can be invoked directly from any of the subclasses
D. can be invoked directly from any of the superclasses

Answer: B, C

`__init__` runs automatically when you instantiate; you never call it directly on the class, but a subclass calls the parent's through `super().__init__()`. It must return None.

66.

The following class definition is given. We want the show() method to invoke the get() method, and then output the value the get() method returns. Which invocation should be used instead of XXX?

```
class Class:
    def __init__(self,val):
        self.val=val
    def get(self):
        return self.val
    def show(self):
        XXX
```

A. print(get(self))
B. print(self.get())
C. print(get())
D. print(self.get(val))

Answer: B

Calling another method on the same object needs `self.` in front - `self.get()`. Without it Python looks for a plain function called `get` and raises `NameError`.

67.

What is the expected output of the following snippet?

```
class X: pass
class Y(X): pass
class Z(Y): pass
X=Z()
Z=Z()
print(isinstance(X,Z), isinstance(Z,X))
```

A. True False
B. True True
C. False False
D. False True

Answer: D

This question is broken. *`Z = Z()` rebinds the name Z from a class to an *instance*, so `isinstance(X, Z)` raises `TypeError: isinstance() arg 2 must be a type`. The code never prints anything, and no option says it raises.*

`Z = Z()` rebinds the name Z from the class to an instance of it, so the class is no longer reachable by that name and `isinstance(X, Z)` gets an object where it needs a type.

68.

Which sentence about the @property decorator is false?

A. The @property decorator should be defined after the method that is responsible for setting an encapsulated attribute.
B. The @property decorator designates a method which is responsible for returning an attribute value.
C. The @property decorator marks the method whose name will be used as the name of the instance attribute.
D. The @property decorator should be defined before the methods that are responsible for setting and deleting an encapsulated attribute.

Answer: A

*`@property` goes on the **getter**, and it has to come first because the setter is then written as `@name.setter`, which refers back to the property the getter created. Defining it afterwards would have nothing to attach to.*

69.

Select the true statement about the __name__ attribute.

A. __name__ is a special attribute, inherent for both classes and instances, and it contains information about the class to which a class instance belongs.
B. __name__ is a special attribute, inherent for both classes and instances, and it contains a dictionary of object attributes.
C. __name__ is a special attribute, inherent for classes and it contains information about the class to which a class instance belongs.
D. __name__ is a special attribute, inherent for classes, and it contains the name of a class.

Answer: D

*`__name__` on a class holds the class's own name as a string. The attribute that tells an *instance* which class it belongs to is `__class__`, and the dictionary of attributes is `__dict__`.*

70.

What is the expected behaviour of the following code? (missing colon after Sub_B(Super))

A. it outputs 0
B. it outputs 1
C. it raises an exception
D. it outputs 2

Answer: C

A missing colon after a `class` header is a `SyntaxError`, so the file never runs.

71.

Assuming that the following inheritance set is in force, which of the following classes are declared properly?

```
class A: pass
class B(A): pass
class C(A): pass
class D(B): pass
```

A. class Class_4(D,A): pass
B. class Class_3(A,C): pass
C. class Class_2(B,D): pass
D. class Class_1(C,D): pass

Answer: A, C

A class list is valid when Python can find one consistent method resolution order. Deriving from a class and its own ancestor in the wrong order is what fails - the more specific class has to come first.

72.

What is the expected behaviour of this code?

```
class Upper:
    def method(self):
        return 'upper'
class Lower(Upper):
    def method(self):
        return 'lower'
Object=Upper()
print(isinstance(Object,Lower), end=' ')
print(Object.method())
```

A. True upper
B. True lower
C. False upper
D. False lower

Answer: C

`Object` is an `Upper`, and a parent is never an instance of its child, so `isinstance` is False. The method called is the one on `Upper`, printing `upper`.

73.

What is the expected behaviour of the following code? (o1.foo missing parentheses)

A. it outputs 1
B. it outputs 3
C. it outputs 6
D. it raises an exception

Answer: D

*`o1.foo` without parentheses is the method *object*, not a call. Using it where a number is expected raises `TypeError`.*

74.

What is the expected behaviour of this code?

```
class Class:
    Variable=0
    def __init__(self):
        self.value=0
object_1=Class()
object_1.Variable+=1
object_2=Class()
object_2.value+=1
print(object_2.Variable+object_1.value)
```

A. it outputs 0
B. it raises an exception
C. it outputs 1
D. it outputs 2

Answer: A

*`object_1.Variable += 1` reads the shared class attribute (0), adds one, and stores the result as a **new instance attribute** on `object_1` only. `object_2.Variable` still reads the class's 0, and `object_1.value` is its own 0, so the sum is 0.*

75.

What is true about Object-Oriented Programming in Python?

A. each object of the same class can have a different set of methods
B. a subclass is usually more specialised than its superclass
C. if a real-life object can be described with a set of adjectives, they may reflect a Python object method
D. the same class can be used many times to build a number of objects

Answer: B, D

Every instance of a class has the same methods - that is what the class defines. A subclass specialises its parent, and one class builds as many objects as you like.

76.

What is true about Python class constructors?

A. there can be more than one constructor in a Python class
B. the constructor must return a value other than None
C. the constructor is a method named __init__
D. the constructor must have at least one parameter

Answer: C, D

The constructor is `__init__`, and its first parameter is the instance, so it always has at least one. It must return None, and a second `def __init__` simply replaces the first - Python has no overloading.

77.

Assuming that the following piece of code has been executed successfully, which of the expressions evaluate to True?

```
class A:
    VarA=1
    def __init__(self):
        self.prop_a=1
class B(A):
    VarA=2
    def __init__(self):
        super().__init__()
        self.prop_b=2
obj_a=A()
obj_aa=A()
obj_b=B()
obj_bb=obj_b
```

A. isinstance(obj_b,A)
B. A.VarA==1
C. obj_a is obj_aa
D. B.VarA==1

Answer: A, B

`obj_b` is a B, and B derives from A, so `isinstance(obj_b, A)` is true. `A.VarA` is still 1 - B shadowing it with 2 does not change the parent. `obj_a` and `obj_aa` are separate objects, so `is` is false.

78.

What is the expected behaviour of this code? Which are true?

```
class Class:
    Var=data=1
    def __init__(self, value):
        self.prop=value
Object=Class(2)
```

A. len(Class.__dict__)==1
B. 'data' in Class.__dict__
C. 'var' in Class.__dict__
D. 'data' in Object.__dict__

Answer: B

The stem says choose two but the bank keys only one. `Var = data = 1` binds both names in the class body, so `'data' in Class.__dict__` is true.

*`Var = data = 1` in the class body binds **both** names on the class, so `'data'` is in `Class.__dict__`. `'var'` is not - names are case-sensitive - and `prop` lives on the instance, not the class.*

79.

A property that stores information about a given class's super-classes is named:

A. __upper__
B. __super__
C. __ancestors__
D. __bases__

Answer: D

`__bases__` holds the tuple of a class's direct base classes.

80.

What is the expected behaviour of this code?

```
class Class:
    Var=0
    def _foo(self):
        Class.Var+=1
        return Class.Var
o=Class()
o._Class_foo()
print(o._Class_foo())
```

A. it raises an exception
B. it outputs 3
C. it outputs 1
D. it outputs 6

Answer: A

*Only a **double** underscore triggers name mangling. `_foo` has one, so it is not renamed and `o._Class_foo()` looks for an attribute that does not exist, raising `AttributeError`. Had the method been `__foo`, the mangled name would be `_Class__foo` - note the two underscores in the middle.*

81.

The __bases__ property contains:

A. base class location (addr)
B. base class objects (class)
C. base class names (str)
D. base class ids (int)

Answer: B

*`__bases__` holds the base **class objects** themselves, not their names or addresses. `SubClass.__bases__[0].__name__` is how you would get a name.*

82.

What is the expected behaviour of this code?

```
class Super:
    def make(self): pass
    def doit(self):
        return self.make()
class Sub_A(Super):
    def make(self):
        return 1
class Sub_B(Super):
    pass
a=Sub_A()
b=Sub_B()
print(a.doit()+b.doit())
```

A. It outputs 0

B. It raises an exception
C. It outputs 1
D. It outputs 2

Answer: B

`Sub_A` overrides `make()` and returns 1, but `Sub_B` inherits the base version, whose body is `pass` - so it returns `None`. `1 + None` raises `TypeError`.

83.

Assuming that the code below has been executed successfully, which of the expressions evaluate to True?

```
class Class:
    var=1
    def __init__(self, value):
        self.prop=value
Object=Class(2)
```

A. 'var' in Class.__dict__
B. 'var' in Object.__dict__
C. len(Object.__dict__) == 1
D. 'prop' in Class.__dict__

Answer: A, C

`var` is defined in the class body so it lives on the class; `prop` is assigned to `self` so it lives on the instance, which therefore has exactly one entry in its `__dict__`.

84.

What is true about Python class constructors?

A. there can be only one constructor in a Python class
B. the constructor cannot be invoked directly under any circumstances
C. the constructor cannot return a result other than None
D. the constructor's first parameter must always be named self

Answer: B, D

`__init__` is invoked for you by instantiation, never called directly, and its first parameter is conventionally `self`. A second `def __init__` replaces the first rather than overloading it.

85.

Assuming that the following inheritance set is in force, which of the following classes are declared properly?

```
class A: pass
class B(A): pass
class C(A): pass
class D(B,C): pass
```

A. class Class_3(A,C): pass
B. class Class_4(C,B): pass
C. class Class_1(D): pass
D. class Class_2(A,B): pass

Answer: A, C

`class Class_3(A, C)` and `class Class_1(D)` both give Python a consistent method resolution order. The failing combinations put a base class before its own subclass, which cannot be linearised.

86.

What is the expected behaviour of this code?

```
class Class:
    Variable=0
    def __init__(self):
```

```
        self.value=0
object_1=Class()
Class.Variable+=1
object_2=Class()
object_2.value+=1
print(object_2.Variable+object_1.value)
```

A. It outputs 2
B. It raises an exception
C. It outputs 1
D. It outputs 0

Answer: C

*`Class.Variable += 1` changes the attribute **on the class**, so every instance sees 1. Compare the near-identical question that writes `object_1.Variable += 1` instead - that creates a private copy on one instance and leaves the class at 0.*

87.

What is the expected behaviour of this code?

```
class Upper:
    def __init__(self):
        self.property='upper'
class Lower(Upper):
    def __init__(self):
        super().__init__()
Object=Lower()
print(isinstance(Object,Lower), end='')
print(Object.property)
```

A. True lower
B. True upper
C. False upper
D. False lower

Answer: B

`Lower` derives from `Upper` and its `__init__` calls `super().__init__()`, so `property` does get set and reads `'upper'`. The object genuinely is a `Lower`, so `isinstance` is True.

88.

What is the expected behaviour of this code?

```
class ClassA:
    var=1
    def __init__(self,prop):
        prop1=prop2=prop
class ClassB(ClassA):
    def __init__(self,prop):
        prop3=prop**2
        super().__init__(prop)
    def __str__(self):
        return 'Object'
Object=ClassA(2)
```

A. ClassA.__module__ == '__main__'
B. __name__ == '__main__'
C. str(Object) == 'Object'
D. len(ClassB.__bases__) == 2

Answer: A, B

This question's key was wrong and is corrected here. *`__str__` is defined on `ClassB`, but `Object` is a `ClassA`, so `str(Object)` gives the default `<__main__.ClassA object ...>` - option C is false. `ClassB.__bases__` is `(ClassA,)`, length 1, so D is false too. The two true statements are A and B.*

`ClassA.__module__` is `'__main__'` when the file is the one being run. Watch the `__str__`: it is defined on `ClassB`, and `Object` is a `ClassA`, so it does not apply.

89.

Scenario: You are developing a game in Python and want to represent different characters as objects. Which OOP concept will you use?

A. Inheritance
B. Encapsulation
C. Polymorphism
D. Abstraction

Answer: C

Unicode is a standard that assigns a number, called a code point, to every character in every writing system, plus encodings such as UTF-8 that store those numbers as bytes.

Domain 4 — Modules and Libraries (17)

90.

Can a module run like regular code?

A. yes, and it can differentiate its behaviour between the regular launch and import
B. it depends on the Python version
C. yes, but it cannot differentiate its behaviour between the regular launch and import
D. no, it is not possible; a module can be imported, not run

Answer: A

A module is just a Python file, so it can be run directly. The `__name__` variable is `'__main__'` when it is run and the module's own name when it is imported, which is exactly how a file tells the two apart.

91.

A file name like services.cpython-36.pyc says that:

A. the interpreter used to generate the file is version 3.6
B. it has been produced by CPython
C. it is the 36th version of the file
D. the file comes from the services.py source file

Answer: A, B, D

The name encodes three things: the source file (`services.py`), the implementation that compiled it (CPython) and that implementation's version (3.6). It is not a revision number.

92.

What can you do if you don't like a long package path like import alpha.beta.gamma.delta.epsilon.zeta?

A. you can make an alias for the name using the alias keyword
B. nothing, you need to come to terms with it
C. you can shorten it to alpha.zeta and Python will find the proper connection
D. you can make an alias for the name using the as keyword

Answer: D

`import a.b.c as short` binds the shorter name. There is no `alias` keyword, and Python will not guess at a shortened path.

93.

What should you put instead of XXX to print out the module name?

```
if __name__ != 'XXX':
    print(__name__)
```

A. main
B. _main
C. __main__
D. _main_

Answer: C

Python sets `__name__` to the string `'__main__'` in the file being run directly, and to the module's own name when it is imported. Note the two underscores on each side.

94.

Files with the suffix .py contain:

A. Python source code
B. backups
C. temporary data
D. semi-compiled Python code

Answer: A

`.py` is Python source. `.pyc` holds the semi-compiled bytecode, which is what D describes.

95.

Package source directories/folders can be:

A. converted into the so-called pycK format
B. packed as a ZIP file and distributed as one file
C. rebuilt to a flat form and distributed as one directory/folder
D. removed as Python compiles them into an internal portable format

Answer: B

A package is a directory tree, and it can be zipped and shipped as one file - which is what a wheel is underneath.

96.

What can you deduce from the line below?

```
x = a.b.c.f()
```

A. import a.b.c should be placed before that line
B. f() is located in subpackage c of subpackage b of package a
C. the line is incorrect
D. the function being invoked is called a.b.c.f()

Answer: A, B, D

The stem says choose two, but the bank keys three (A, B and D). All three are in fact correct, so the count in the stem is the error.

`x = a.b.c.f()` calls `f` from subpackage `c`, inside `b`, inside package `a`, so the full dotted path is the function's name and the package has to be imported first for the lookup to resolve.

97.

Assuming that the code below has been executed successfully, which of the following expressions will always evaluate to True?

```
import random
random.seed(1)
v1=random.random()
random.seed(1)
v2=random.random()
```

A. v1 == v2
B. v1 > v2
C. len(random.sample([1,2,3],2)) > 2
D. random.choice([1,2,3]) >= 1

Answer: A, D

Seeding with the same value makes the sequence repeat, so `v1 == v2` always holds. `random.choice` on `[1,2,3]` can only return one of those, all >= 1. `sample` cannot return more items than you asked for.

98.

Which one of the platform module functions should be used to determine the underlying platform name?

A. platform.python_version()
B. platform.processor()
C. platform.platform()
D. platform.uname()

Answer: C

`platform.platform()` returns the whole platform string. `uname()` returns a structured tuple, and `processor()` only the CPU.

99.

What is the expected output of the following code?

```
import sys
import math
b1=type(dir(math)[0]) is str
b2=type(sys.path[-1]) is str
print(b1 and b2)
```

A. FALSE
B. None
C. TRUE
D. 0

Answer: C

`dir()` returns a list of names as strings and `sys.path` is a list of directory strings, so both checks are true and and gives True. (The option is spelled TRUE here; Python prints `True`.)

100.

With regards to the directory structure, select the proper forms of the directives in order to import module_a.

A. from pypack import module_a
B. import module_a from pypack
C. import module_a
D. import pypack.module_a

Answer: A, D

Either name the package on the way in - `from pypack import module_a` - or import the full dotted path. A bare `import module_a` only works if the module's own directory is on the path.

101.

A Python module named pymod.py contains a function named python(). Which of the following snippets will let you invoke the function?

A. import pymod; pymod.python()
B. from pymod import python; python()
C. from pymod import *; pymod.python()
D. import python from pymod; python()

Answer: A, B

`import pymod` binds the module, so the function is reached as `pymod.python()`. `from pymod import python` binds the function itself, so it is called bare. Option C mixes the two and would fail.

102.

What is true about Python packages?

A. a package is a single file whose name ends with the .pa extension
B. a package is a group of related modules
C. the __name__ variable always contains the name of a package
D. the .pyc extension is used to mark semi-compiled Python packages

Answer: B, C

*A package is a group of related modules in a directory. `__name__` holds the *module's* name - and `'__main__'` in the file being run - so it is not always a package name; `.pyc` marks compiled modules, not packages.*

103.

A Python module named pymod.py contains a variable named pymod. Which of the following snippets will let you access the variable?

A. import pymod; pymod.pyvar = 1
B. import pymod from pymod; pymod = 1
C. from pymod import pymod; pymod()
D. from pymod import pymod; pymod = 1

Answer: C, D

`from pymod import pymod` binds the module's variable into your namespace, which is how you reach it. Note that rebinding that name afterwards affects only your copy, not the module's.

104.

Assuming that the code below has been executed successfully, which of the following expressions will always evaluate to True?

```
import random
v1=random.random()
v2=random.random()
```

A. v1 == v2
B. v1 < 1
C. random.choice([1,2,3]) > 0
D. len(random.sample([1,2,3],1)) > 2

Answer: B, C

`random.random()` returns a float in [0.0, 1.0), so it is always below 1, and `choice` on `[1,2,3]` always returns one of those. Two unseeded calls are not expected to be equal, and `sample(seq, 1)` returns exactly one item.

105.

With regards to the directory structure, select the proper forms of the directives in order to import module_b.

A. from pyback.upper import module_b
B. import pyback.upper.module_b
C. import upper.module_b
D. import module_b

Answer: B

The stem says choose two but the bank keys only one.

The full dotted path always works: `import pyback.upper.module_b`. A partial path like `upper.module_b` only resolves if that directory is itself on the import path.

106.

What is the expected behaviour of this code?

```
import sys
b1=type(dir(sys)) is str
b2=type(sys.path[-1]) is str
print(b1 and b2)
```

A. FALSE
B. 0
C. None
D. TRUE

Answer: A

This question's key is malformed and is corrected here. *It reads `FALSE` - the option *text* rather than a letter, almost certainly Excel coercing "False" into a boolean. It maps to option A.*

*`dir(sys)` returns a **list**, not a str, so `b1` is False and `b1 and b2` is False without even evaluating `b2`. That short-circuit is why `and` stops at the first falsy operand.*

Domain 5 — Debugging and Error Handling (16)

107.

What is the expected output of the following snippet?

```
s='abc'
for i in len(s):
    s[i]=s[i].upper()
print(s)
```

A. abc
B. The code will cause a runtime exception
C. ABC
D. 123

Answer: B

`len(s)` is an integer and integers are not iterable, so `for i in len(s)` raises `TypeError` before anything else happens. You want `for i in range(len(s))` - and even then, strings are immutable so `s[i] = ...` would fail too.

108.

What is the expected behaviour of the following snippet?

```
def a(l,l):
    return l[l]
print(a(0,[1]))
```

A. cause a runtime exception
B. print 1
C. print 0, [1]
D. SyntaxError

Answer: D

*Two parameters with the same name is a `SyntaxError`, caught when the file is compiled - so this fails before any line runs. That is why D is right and A (a *runtime* exception) is not.*

109.

How to handle an exception and assign it to variable e?

A. except Exception (e):
B. except e = Exception:
C. except Exception as e:
D. such an action is not possible in Python

Answer: C

`except SomeError as e:` binds the exception instance to e. The comma form was Python 2 and is now a syntax error.

110.

What is the expected output of the following code?

```
lst=[x for x in range(5)]
lst=list(filter(lambda x: x%2==0, lst))
print(len(lst))
```

A. 2
B. The code will cause a runtime exception

C. 1
D. SyntaxError

Answer: A

This question is broken. *`[x for x in range(5)]` is `[0,1,2,3,4]`; filtering for `x % 2 == 0` keeps 0, 2 and 4, so `len` is **3**. No option offers 3.*

`range(5)` is 0 to 4. Keeping the even ones leaves 0, 2 and 4, so the length is 3. Remember that 0 is even.

111.

What is the expected behaviour of the following code?

```
def unclear(x):
    if x%2==1:
        return 0
print(unclear(1)+unclear(2))
```

A. print 0
B. cause a runtime exception
C. prints 3
D. print an empty line

Answer: B

The function returns 0 only when the argument is odd; with an even argument it falls off the end and returns None. `0 + None` raises `TypeError`. A function with a conditional `return` returns None on every path you did not cover.

112.

If you need to serve two different exceptions called Ex1 and Ex2 in one except branch, you can write:

A. except Ex1 Ex2:
B. except (Ex1, Ex2):
C. except Ex1, Ex2:
D. except Ex1+Ex2:

Answer: B

Multiple exception types go in a parenthesised tuple. The bare comma form was Python 2 syntax for binding the instance and is now a syntax error.

113.

What is the expected behaviour of the following code? (try/except missing colon)

A. it outputs 3
B. it outputs 1
C. the code is erroneous and it will not execute
D. it outputs 2

Answer: C

A missing colon is a `SyntaxError`, raised when the file is compiled, so nothing runs at all and no output appears.

114.

What is the expected behaviour of the following code?

```
string=str(1/3)
dummy=''
for character in strong:
    dummy=character+dummy
print(dummy[-1])
```

A. it raises an exception
B. it outputs 0
C. it outputs 3
D. it outputs None

Answer: A

`strong` is a typo for `string`, and the name was never assigned, so the loop raises `NameError` before anything prints. Python only catches this when the line actually runs.

115.

What is the expected behaviour of this code?

```
my_list=[i for i in range(5)]
m=[my_list[i] for i in range(4,0,-1)] if my_list[i]%2!=0]
print(m)
```

A. it outputs [1,3]
B. the code is erroneous and it will not execute
C. it outputs [3,1]
D. it outputs [4,2,0]

Answer: B

The comprehension has an unbalanced bracket, so it is a `SyntaxError` and nothing runs. Read the brackets rather than the logic when a snippet looks odd.

116.

Which of the following snippets will execute without raising any unhandled exceptions?

A. try: print(float('1e1')) except (ValueError,NameError): print(float('1a1')) else: print(float('101'))
B. try: print(1/1) except: print(2/1) else: print(3/0)
C. try: print(1/0) except ValueError: print(1/1) else: print(1/2)
D. try: print(0/1) except: print(1/1) else: print(2/1)

Answer: A, D

A works because `float('1e1')` is valid, so no exception occurs and the `else` runs `float('101')`, also valid. D works because `0/1` is fine. B ends with `3/0` in its `else`, and C's `except ValueError` cannot catch the `ZeroDivisionError` that `1/0` raises.

117.

What is the expected behaviour of this code?

```
my_list=[i for i in range(5,0,-1)]
m=[my_list[i] for i in range(5)] if my_list[i]%2==0]
print(m)
```

A. it outputs [4,2]
B. it outputs [2,4]
C. it outputs [0,1,2,3,4]
D. the code is erroneous and it will not execute

Answer: D

The comprehension has an unmatched `]`, so this is a `SyntaxError` and nothing runs at all. When a snippet looks strange, count the brackets before reasoning about the logic.

118.

What is the expected behaviour of this code?

```
the_list='1,2 3'.split()
the_string=''.join(the_list)
print(the_string.insight())
```

A. it raises an exception
B. it outputs nothing
C. it outputs True
D. it outputs False

Answer: A

`.split()` and `.join()` both work fine, but `.insight()` is not a string method - it does not exist - so the code raises `AttributeError` before `print` gets anything. Nothing is output because the program stops there.

119.

What is the expected behaviour of this code?

```
s='2A'
try:
    n=int(s)
except ValueError:
    n=2
except ArithmeticError:
    n=1
except:
    n=0
print(n)
```

A. it outputs 0
B. the code is erroneous and it will not execute
C. it outputs 1
D. it outputs 2

Answer: D

`int('2A')` raises `ValueError`, and handlers are tried in order, so the first matching one wins and n becomes 2. The later branches never run.

120.

Which of the following snippets will execute without raising any unhandled exceptions?

A. try: print(0/0) except: print(0/1) else: print(0/2)
B. try: print(int('0')) except NameError: print('0') else: print(int(''))
C. import math try: print(math.sqrt(-1)) except: print(math.sqrt(0)) else: print(math.sqrt(1))
D. try: print(float('1e1')) except (NameError, SystemError): print(float('1a1')) else: print(float('1c1'))

Answer: A, C

*A and C both raise inside the `try` and catch it with a bare `except`, whose body succeeds - so nothing escapes. B and D succeed in the `try`, which means their `else` runs, and both `else` bodies raise (`int('')` and `float('1c1')`) with no handler left. An `else` block is **not** protected by the `except` above it.*

121.

What is the expected behaviour of this code?

```
my_list=[1,2,3]
try:
    my_list[3]=my_list[2]
except BaseException as error:
    print(error)
```

A. it outputs error
B. it outputs <class 'IndexError'>
C. it outputs list assignment index out of range
D. the code is erroneous and it will not execute

Answer: C

*`print(error)` prints the exception's *message*, not its class. `my_list[3]` is past the end of a three-element list, so the message is `list assignment index out of range`. Printing `type(error)` is what would give you the class.*

122.

What is the expected behaviour of this code?

```
class E(Exception):
    def __init__(self,message):
        self.message=message
    def __str__(self):
        return 'it''s nice to see you'
```

```
try:
    print('I feel fine')
    raise Exception('what a pity')
except E as e:
    print(e)
else:
    print('the show must go on')
```

A. the string 'it's nice to see you' will be seen
B. the string 'the show must go on' will be printed
C. the string 'I feel fine' will be printed
D. nothing will be printed

Answer: C

`raise Exception(...)` raises the base class, but the handler only catches E, so it does not match and the exception is unhandled. `'I feel fine'` has already printed by then; the `else` never runs because the `try` did not complete.

Domain 6 — File Handling (11)

123.

There is a stream named s open for writing. What option will you select to write a line to the stream?

A. s.write('Hello\n')
B. write(s,'Hello')
C. s.writeln('Hello')
D. s.writeln('Hello')

Answer: A

Options C and D are identical, so one is a typo. It does not affect the answer: there is no `writeln` in Python at all.
File objects have `write()`, which adds nothing of its own - you supply the newline. There is no `writeln` in Python.

124.

You are going to read just one character from a stream called s. Which statement would you use?

A. ch = read(s,1)
B. ch = s.input(1)
C. ch = input(s,1)
D. ch = s.read(1)

Answer: D

`read(n)` reads at most n characters, so `s.read(1)` gets exactly one. `input()` reads from the keyboard, not from a file object.

125.

What can you deduce from the following statement?

```
str = open('file.txt','rt')
```

A. str is a string read in from the file named file.txt
B. a newline character translation will be performed during the reads
C. if file.txt does not exist, it will be created
D. the opened file cannot be written with the use of the str variable

Answer: B, D

`'rt'` is read + text mode. Text mode translates newlines on the way in, and read mode means the object cannot be written through. Opening for reading never creates a missing file - it raises `FileNotFoundError`.

126.

What does the open() function return?

A. an integer value identifying an opened file
B. an error code (0 means success)
C. a stream object
D. always None

Answer: C

`open()` returns a file object, also called a stream. Errors are reported by raising, not by a return code.

127.

If S is a stream open for reading, what do you expect from the following invocation?

```
c = s.read()
```

A. one line of the file will be read and stored in the string called c
B. the whole file content will be read and stored in the string called c
C. one character will be read and stored in the string called c
D. one disk sector (512 bytes) will be read and stored in the string called c

Answer: B

*`read()` with no argument consumes the **whole** remaining file into one string. `read(1)` gets one character and `readline()` gets one line.*

128.

You are going to read 16 bytes from a binary file into a bytearray called data. Which lines would you use?

A. data=bytearray(16); bf.readinto(data)
B. data=binfile.read(bytearray(16))
C. bf.readinto(data=bytearray(16))
D. data=bytearray(binfile.read(16))

Answer: A, D

`readinto()` fills a buffer you already made; `read(n)` returns fresh bytes you can wrap in a `bytearray`. Both give you 16 bytes - the difference is who allocates.

129.

What is the expected behaviour of this code?

```
try:
    f=open('non_existing_file','w')
    print(1,end=' ')
    s=f.readline()
    print(2,end=' ')
except IOError as error:
    print(3,end=' ')
else:
    f.close()
    print(4,end=' ')
```

A. 124
B. 1234
C. 24
D. 13

Answer: D

This question's key was wrong and is corrected here. *The bank keys `124`; running it prints `1  3`.*

*The bank keys **124**; it actually prints **1 3**. Opening in `'w'` mode succeeds and creates the file, so `print(1)` runs. Then `readline()` on a write-only handle raises `io.UnsupportedOperation`, which subclasses `OSError` and so IS caught by `except IOError`, printing 3. The `else` never runs, because `else` means the `try` finished without an exception.*

130.

What is the expected behaviour of this code?

```
try:
    f=open('existing_text_file','w')
    d=f.readlines()
    print(len(d))
    f.close()
except IOError:
    print(-1)
```

A. the length of the first line from the file
B. -1
C. the number of lines contained inside the file
D. the length of the last line from the file

Answer: B

This question's key is unusable and is corrected here. *It reads `0 (none of the above)`, which is not one of the four options. It prints **-1**. `'w'` mode truncates the file, then `readlines()` on a write-only handle raises `io.UnsupportedOperation` - a subclass of `OSError` - so the handler catches it.*

131.

Which of the following statements are true?

A. if invoking open() fails, an exception is raised
B. open() requires a second argument
C. open() is a function which returns an object that represents a physical file
D. instd, outstd, errstd are the names of pre-opened streams

Answer: A, C

`open()` raises on failure rather than returning an error code, and what it returns represents the file. The mode argument is optional and defaults to `'r'`; the pre-opened streams are `stdin`, `stdout` and `stderr`.

132.

What is the expected behaviour of this code?

```
try:
    f=open('non_zero_length_existing_text_file','rt')
    d=f.read(1)
    print(len(d))
    f.close()
except IOError:
    print(-1)
```

A. 1
B. 0
C. an error value corresponding to file not found
D. 1

Answer: A

Options A and D are identical (1), so one is a typo. The answer is 1: `read(1)` on a non-empty existing file returns one character, and `len` of that is 1.

`read(1)` returns one character from a non-empty file, so `len(d)` is 1. The `except` branch would only run if the file were missing.

133.

Story: Sarah is writing a Python program that reads data from a file. Which of the following modes should she use to open the file for reading?

A. 'w'
B. 'r'
C. 'a'

D. ’x’

Answer: B

A missing colon after a `class` header is a `SyntaxError`, so the file never runs.